

Manejo de Conectores

Tabla de contenido

Introducción	4
El desfase objeto-relacional.	6
Protocolos de acceso a bases de datos	7
Conectores	10
Arquitectura JDBC.....	15
Conectores tipo 1 y tipo 2.....	17
Tipo 1: puente JDBC-ODBC.....	17
Tipo 2: controlador parcial de Java.....	18
Conectores tipo 3 y tipo 4.....	19
Tipo 3: JDBC-Net pure Java driver.....	19
Tipo 4: Protocolo nativo.	19
Comparación de los distintos tipo de conectores:.....	19
Conexión a una Base de Datos	20
Requisitos previos:.....	20
Creación de la base de datos	22
Drivers de conexión (Oracle y MySQL).....	23
Driver de MySQL.....	23
Driver de Oracle	23
Instalar el conector de la base de datos	25
MySQL.....	25
Oracle.....	26
JDBC conexión a una base de datos.....	28
Realización de consultas.....	31
SELECT	33
UPDATE, DELETE, INSERT	34
Ejecución de sentencias DML.....	36
Ejecución de Procedimientos Almacenados.....	37
Ejecución de procedimientos almacenados en la base de datos MySQL	38
Ejecución de procedimientos almacenados en la base de datos Oracle	41
Sentencias Preparadas.....	43
Captura de errores y liberación de recursos.....	48
Liberación de recursos.....	48
Gestión de errores	48
Try-with-resources	49
Transacciones.....	52
Commit y Rollback	56
Consultas al diccionario de datos.....	58
Recuperación y modificación de valores de ResultSet	61
Pool de conexiones.....	66
Problemas en la creación de conexiones.....	66
Pool de conexiones.....	67
Ejemplo de pool de conexiones Mysql.....	69
Ejemplo uso de un pool de conexiones en Java con Apache Commons DBCP.....	69
Ejemplo de pool de conexiones Oracle.....	71
Operaciones: ejecución de consultas	74

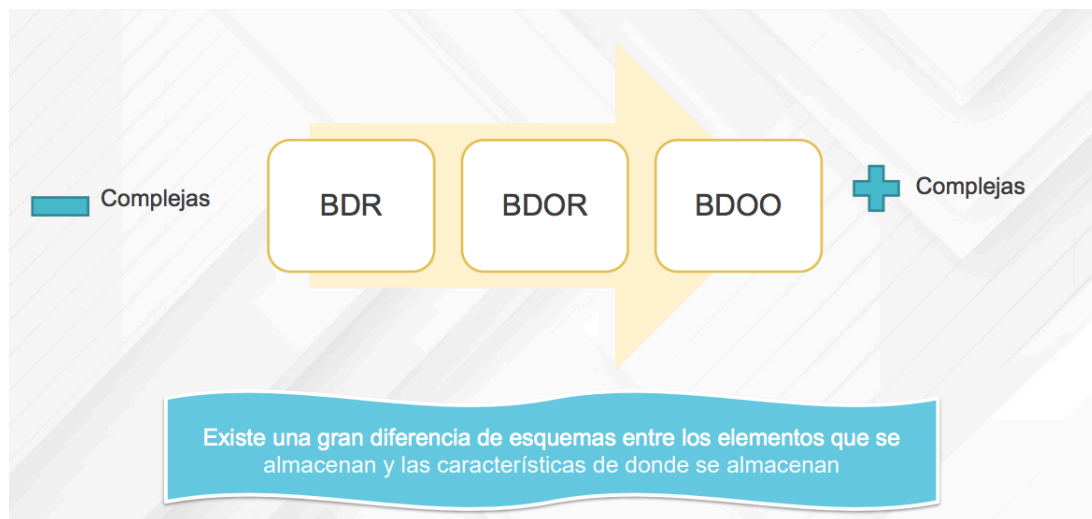
Unidad 2

Módulo – Acceso a Datos
Manejo de Conectores

Excepciones y cierre de conexiones	75
Excepciones	76
Cierre de conexiones	77
EJERCICIOS BLOQUE 1 (SQL – DDL)	78
EJERCICIOS BLOQUE 2 (SQL – DML)	78
PRACTICA FINAL 1	79
PRACTICA FINAL 2	81
PRACTICA EXTRA	82

Introducción

Actualmente, las bases de datos relacionales constituyen el sistema de almacenamiento probablemente más extendido, aunque otros sistemas de almacenamiento de la información se estén abriendo paso poco a poco.



Una [base de datos relacional](#) se puede definir, de una manera simple, como aquella que presenta la información en tablas con filas y columnas.

Una [tabla o relación](#) es una colección de objetos del mismo tipo (filas o tuplas).

En cada tabla de una base de datos se elige una clave primaria para identificar de manera unívoca a cada fila de la misma.

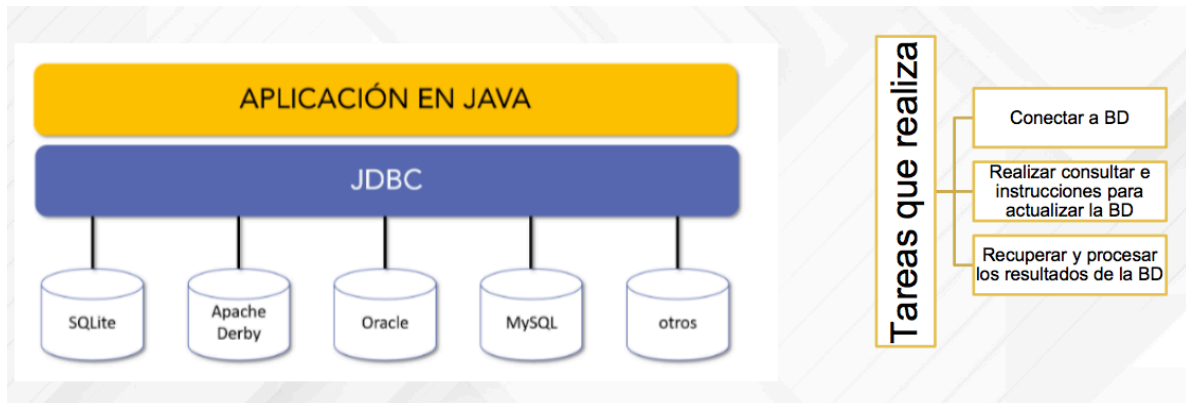
El sistema gestor de bases de datos, en inglés conocido como: [Database Management System \(DBMS\)](#) gestiona el modo en que los datos se almacenan, mantienen y recuperan.

En el caso de una base de datos relacional, el sistema gestor de base de datos se denomina: [Relational Database Management System \(RDBMS\)](#).

Tradicionalmente, la programación de bases de datos ha sido como una torre de Babel: gran cantidad de productos de bases de datos en el mercado y cada uno "hablando" en su lenguaje privado con las aplicaciones.

Java, mediante **JDBC (Java Database Connectivity)**, permite simplificar el acceso a bases de datos relacionales, proporcionando un lenguaje mediante el cual las aplicaciones pueden comunicarse con motores de bases de datos. Se desarrolló este API para el acceso a bases de datos, con tres objetivos principales en mente:

- Ser un API con soporte de SQL: poder construir sentencias SQL e insertarlas dentro de llamadas al API de Java.
- Aprovechar la experiencia de los API's de bases de datos existentes.
- Ser lo más sencillo posible.



Indica si la afirmación es verdadera o falsa:

JDBC tiene como uno de sus principales objetivos facilitar el acceso a bases de datos relacionales.

- Verdadero
- Falso

El desfase objeto-relacional.

El desfase objeto-relacional, también conocido como impedancia objeto-relacional, consiste en la diferencia de aspectos que existen entre la programación orientada a objetos y la base de datos. Estos aspectos se puede presentar en cuestiones como:

- **Lenguaje de programación:** el programador debe conocer el lenguaje de programación orientado a objetos (POO) y el lenguaje de acceso a datos.
- **Tipos de datos:** en las bases de datos relacionales siempre hay restricciones en el uso de tipos, mientras que la programación orientada a objetos utiliza tipos de datos mas complejos.
- **Paradigma de programación:** en el proceso de diseño y construcción del software se tiene que hacer una traducción del modelo orientado a objetos de clases al modelo Entidad-Relación (E/R) puesto que el primero maneja objetos y el segundo maneja tablas y tuplas (o filas), lo que implica que se tengan que diseñar dos diagramas diferentes para el diseño de la aplicación.

El **modelo relacional** trata con **relaciones** y conjuntos debido a su **naturaleza matemática**. Sin embargo, el **modelo de POO** trata con **objetos** y las asociaciones entre ellos. Por esta razón, el problema entre estos dos modelos surge en el momento de querer persistir los objetos de negocio.

La escritura (y de manera similar la lectura) mediante JDBC implica:

- Abrir una conexión.
- Crear una sentencia en SQL.
- Copiar todos los valores de las propiedades de un objeto en la sentencia, ejecutarla y así almacenar el objeto.

Esto es sencillo para un caso simple, pero complicado si el objeto posee muchas propiedades, o bien se necesita almacenar un objeto que a su vez posee una colección de otros elementos. Se necesita crear mucho más código, además del tedioso trabajo de creación de sentencias SQL.

Este problema es lo que denominábamos **impedancia Objeto-Relacional**, o sea, el conjunto de dificultades técnicas que surgen cuando una base de datos relacional se usa en conjunto con un programa escrito en POO.

Como ejemplo de **desfase objeto-relacional**, podemos poner el siguiente: supón que tienes un objeto "Agenda Personal" con un atributo que sea una colección de objetos de la clase "Persona". Cada persona tiene un atributo "teléfono". Al transformar este caso a relacional, se ocuparía más de una tabla para almacenar la información, conllevando varias sentencias SQL y bastante código.

Indica si la afirmación es verdadera o falsa:

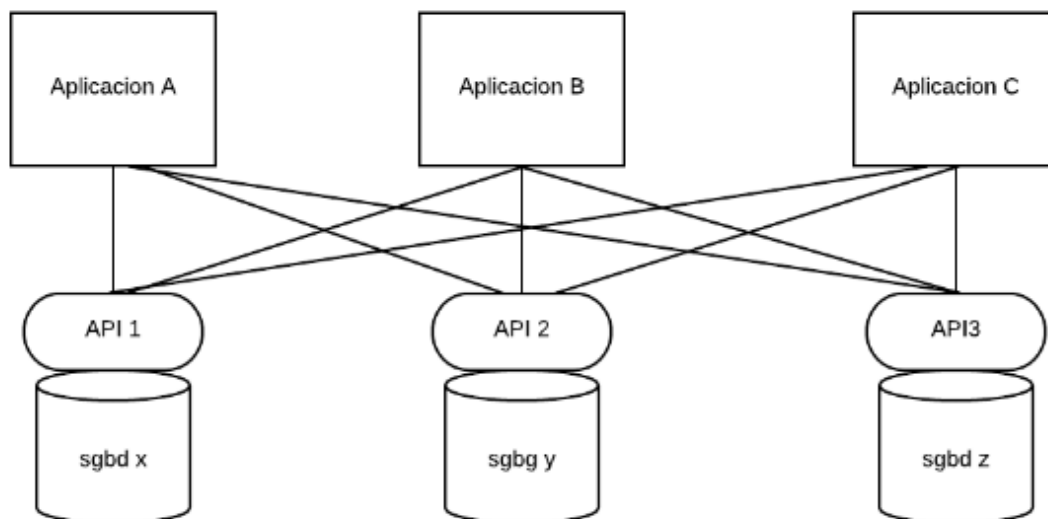
El desfase objeto-relacional tiene que ver con que el paradigma de orientación a objetos tiene una naturaleza matemática y el modelo relacional no.

- c. Verdadero
- d. Falso

Protocolos de acceso a bases de datos

Inicialmente, cada empresa desarrolladora de un SGBD implementaba soluciones propietarias específicas para su sistema, pero pronto se dieron cuenta de que colaborando conjuntamente podían sacar mayor rendimiento y avanzar mucho más rápidamente.

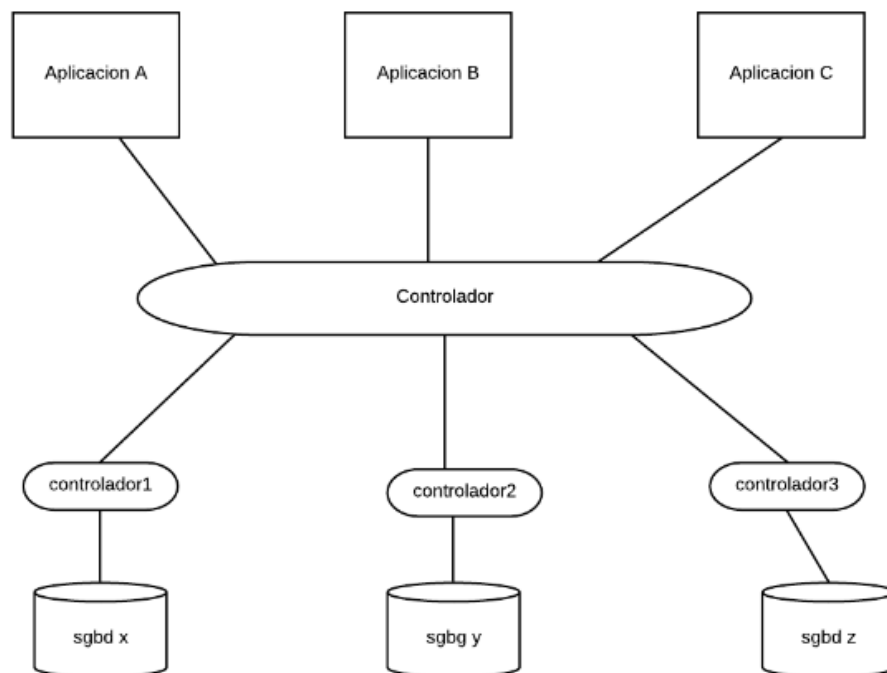
Cada SGBD tiene su propia conexión y su propio API.



La llegada del **ODBC** representó un avance sin precedentes en el camino hacia la interoperabilidad entre bases de datos y lenguajes de programación. La mayoría de empresas desarrolladoras de sistemas gestores de bases de datos incorporaron los drivers de conectividad a las utilidades de sus sistemas y los lenguajes de programación más importantes desarrollaron bibliotecas específicas para soportar el **API ODBC**.

Aunque la industria aceptó ODBC como medio principal para acceso a bases de datos en Windows, la verdad es que no se introduce bien en el mundo Java, debido a la complejidad que presenta ODBC, y que entre otras cosas ha impedido su transición fuera del entorno Windows.

Sistema de conexión **ODBC** configurado usando diferentes controladores (drivers) y un API estándar.



JDBC (Java Data Base Connectivity) se trata de un API bastante similar a **ODBC** en cuanto a funcionalidad, implementado específicamente para usar con el lenguaje Java, adaptado a las especificidades de Java. Es decir, la funcionalidad se encuentra capsulada en clases (ya que Java es un lenguaje totalmente orientado a objetos) y además, no depende de ninguna plataforma específica, de acuerdo con la característica multiplataforma defendida por Java. La idea en el desarrollo de **JDBC** era intentar ser tan sencillo como fuera posible, pero proporcionando a los desarrolladores la máxima flexibilidad.

Los desarrolladores de Los Sistemas Gestores de Bases de Datos proporcionan la implementación de esa interfaz (Drivers).

JDBC es similar en estructura a **ODBC**. Una aplicación JDBC está compuesta de varias capas, como se muestra en la figura:



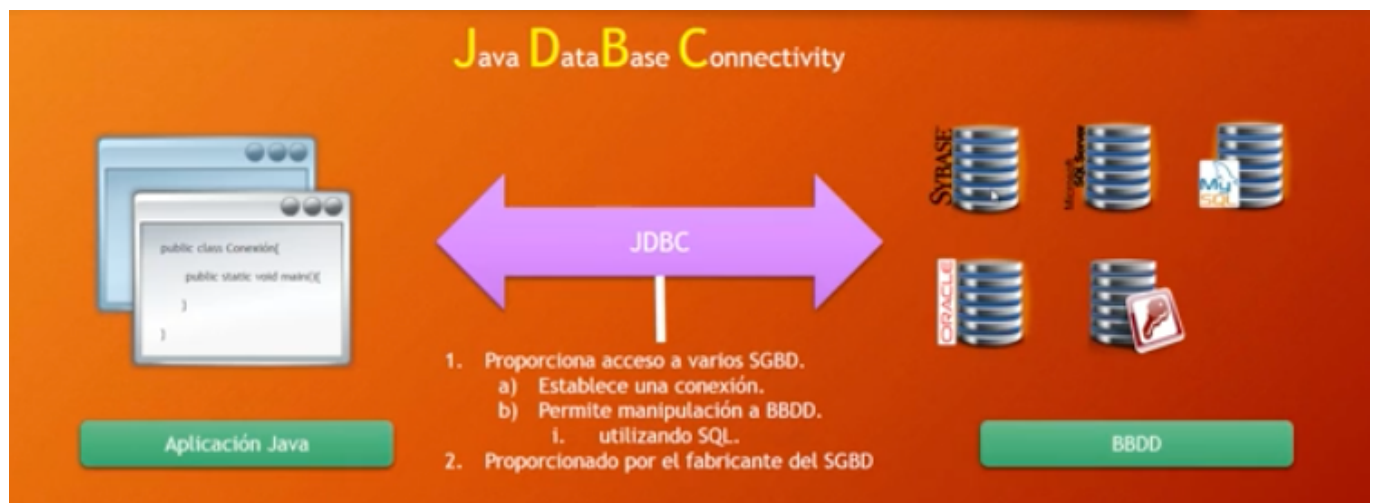
La capa superior en este modelo es la aplicación **Java**. Las aplicaciones **Java** son portátiles: puede ejecutar una aplicación Java sin modificaciones en cualquier sistema que tenga instalado una maquina virtual java (Java runtime environment).

Una aplicación **Java** que utiliza **JDBC** puede comunicarse con muchas bases de datos con pocas modificaciones, si es que las hay. Al igual que **ODBC**, **JDBC** proporciona una manera consistente de conectarse a una base de datos, ejecutar comandos y recuperar los resultados. Al igual que **ODBC**, **JDBC** no impone un lenguaje de comando común: puede usar la sintaxis específica de Oracle cuando está conectado a un servidor Oracle y la sintaxis específica de MySQL cuando está conectado a un servidor MySQL.

La clase **JDBC DriverManager** es responsable de localizar un controlador JDBC que necesita la aplicación. Cuando una aplicación cliente solicita una conexión de base de datos, la solicitud se expresa en forma de una URL (Uniform Resource Locator). Una URL de JDBC es similar a las URL que utiliza con un navegador web. Para conectarnos a la base de datos Oracle, por ejemplo: jdbc:oracle:thin:ejemplo/ejemplo@localhost:1521:XE

A medida que cada controlador se carga en una Máquina Virtual de Java (VM), se registra con el **DriverManager JDBC**. Cuando una aplicación solicita una conexión, el **DriverManager** pregunta a cada Controlador si puede conectarse a la base de datos especificada en la URL dada. Tan pronto como encuentra un controlador adecuado, la búsqueda se detiene y el controlador intenta establecer una conexión con la base de datos. Si el intento de conexión falla, el Controlador lanzará una **SQLException** a la aplicación. Si la conexión se completa con éxito, el controlador crea un objeto de conexión y lo devuelve a la aplicación.

Conectores

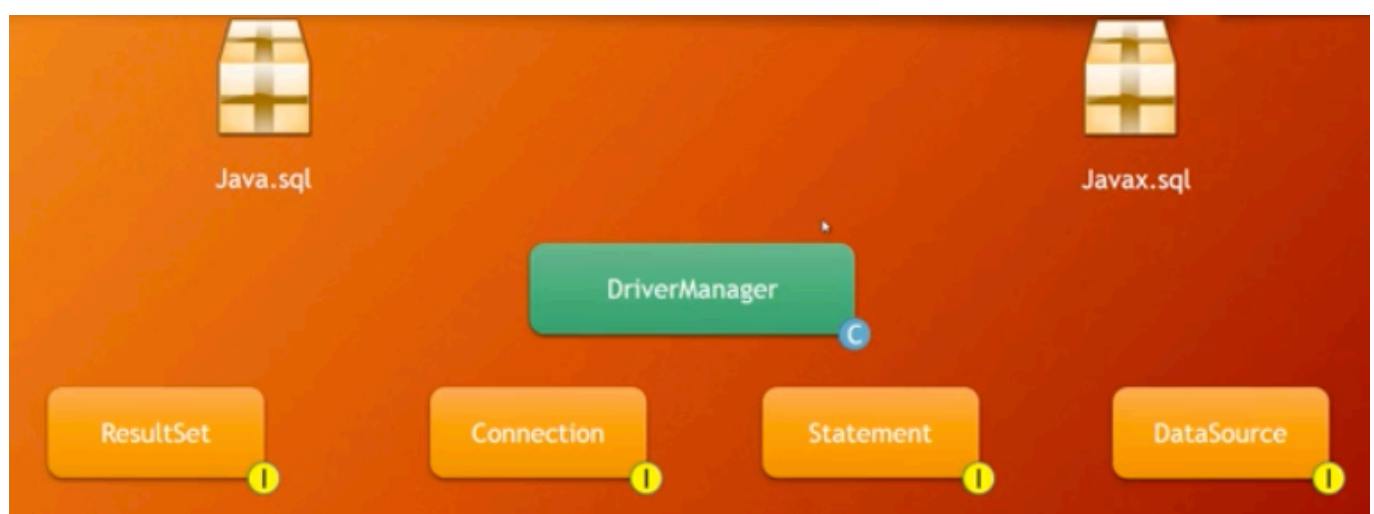


Un **conector o driver** es un conjunto de clases encargadas de implementar los interfaces del API y acceder a la base de datos.

Para poder conectarse a una base de datos y lanzar consultas, una aplicación necesita tener un **driver** adecuado. Un **conector** suele ser un fichero **.jar** que contiene una implementación de todas las interfaces del **API JDBC**.

Cuando se construye una aplicación de base de datos, **JDBC** oculta lo específico de cada base de datos, de modo que el programador se ocupe sólo de su aplicación.

El **conector** lo proporciona el fabricante de la base de datos o bien un tercero.

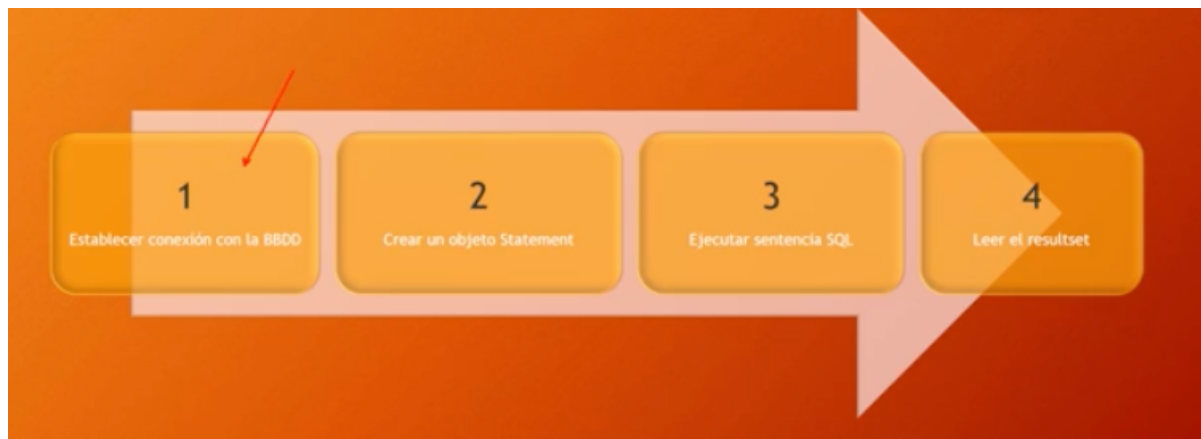


JDBC. Clases a manejar.

El código de nuestra aplicación no depende del driver, puesto que trabajamos mediante los paquetes **java.sql** y **javax.sql**.

JDBC ofrece las clases e interfaces para:

- Establecer una conexión a una base de datos.
- Ejecutar una consulta.
- Procesar los resultados.



JDBC. Proceso acceso a BBDD



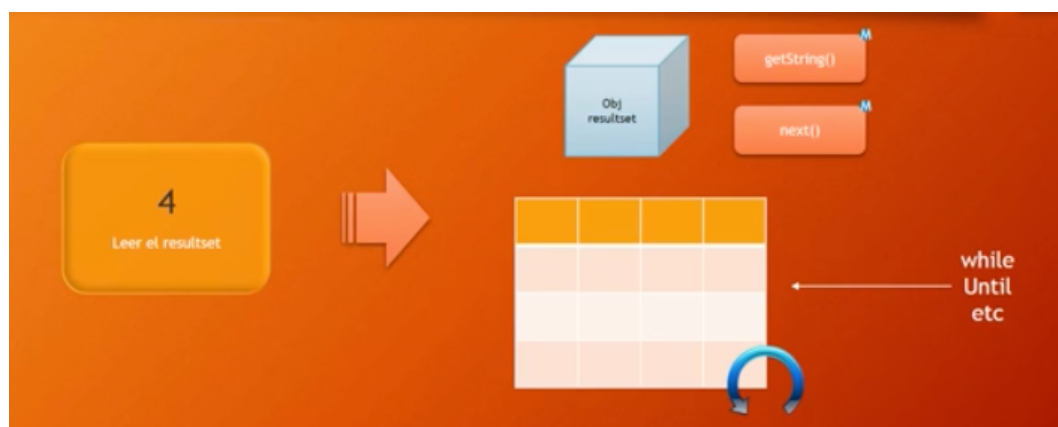
Paso 1. Establecer conexión con la BBDD



Paso 2. Crear Objeto Statement



Paso 3. Ejecutar sentencia SQL



Paso 4. Leer el resultSet

A continuación algunas de las clases e interfaces:

CLASE E INTERFAZ	DESCRIPCIÓN
Driver	Permite conectarse a una BD: cada gestor de BD requiere un driver distinto
DriverManager	Permite gestionar todos los drivers instalados en el sistema
DriverPropertyInfo	Proporciona información sobre un driver
Connection	Representa una conexión con una BD. Una aplicación puede tener más de una conexión
DatabaseMetadata	Proporciona información acerca de una BD (tablas que contiene, tipos de datos, etc)
Statement	Permite ejecutar sentencias SQL sin parámetros
PreparedStatement	Permite ejecutar sentencias SQL con parámetros de entrada
CallableStatement	Permite ejecutar sentencias SQL con parámetros de entrada y salida como llamadas a procedimientos

Ejemplo que establece conexión con una base de datos Oracle:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConsultaSencilla {

    public static void main(String[] args) {

        String urljdbc = "jdbc:oracle:thin:@localhost:1521:XE";
        String usuario = "usuario";
        String password = "tu_password"; // <-- reemplaza con la contraseña real

        try (Connection conexion = DriverManager.getConnection(urljdbc, usuario, password);
            Statement stmt = conexion.createStatement();
            ResultSet rset = stmt.executeQuery("SELECT empno, ename, job FROM emp ORDER BY 1")) {

            while (rset.next()) {
                System.out.println("Empleado número " + rset.getString(1) +
                                   ", nombre " + rset.getString(2) +
                                   ", oficio " + rset.getString(3));
            }

        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

En principio, todos los conectores deben ser compatibles con [ANSI SQL-2 Entry Level](#) ([ANSI SQL-2](#) se refiere a los estándares adoptados por el [American National Standards Institute](#) en 1992. [Entry Level](#) se refiere a una lista específica de capacidades de SQL.) Los desarrolladores de drivers pueden establecer que sus conectores conocen estos estándares.

Hay cuatro tipos de drivers JDBC más usados: [Tipo 1](#), [Tipo 2](#), [Tipo 3](#) y [Tipo 4](#), que veremos en apartados posteriores.

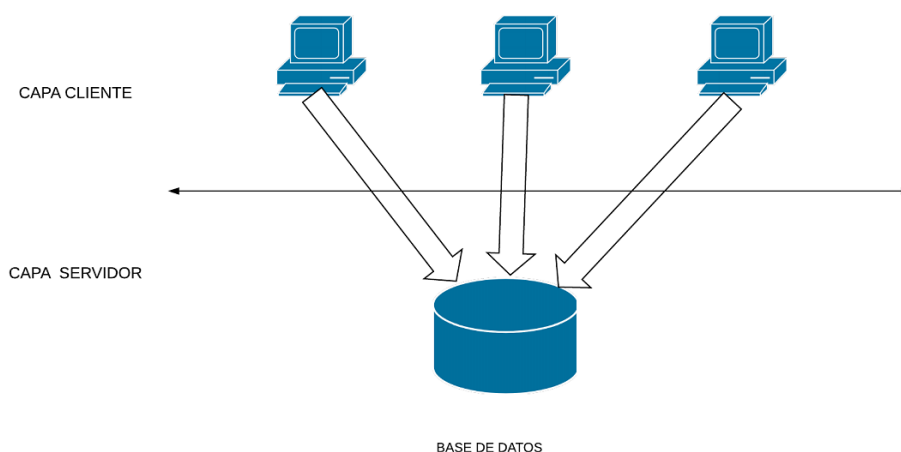
Si quieres saber más de las Base de datos con información sobre [JDBC](#) y tecnologías de base de datos de [Oracle](#), visita:

<https://www.oracle.com/java/technologies/javase/javase-tech-database.html>

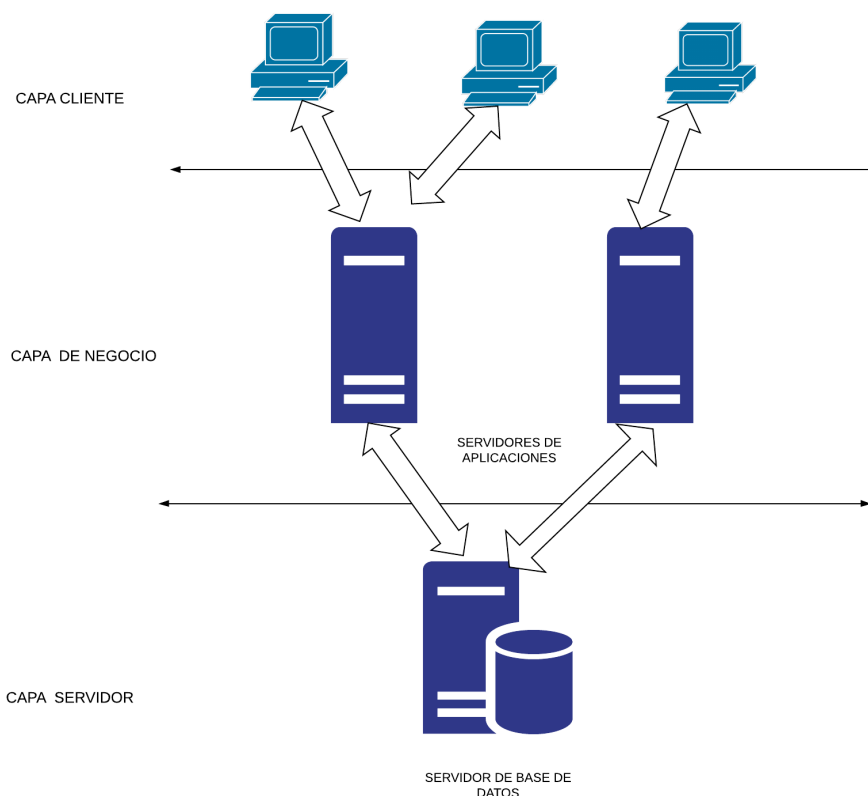
Arquitectura JDBC

En el **modelo de dos capas**, una aplicación se comunica directamente a la fuente de datos. Esto necesita un **conector JDBC** que pueda comunicar con la fuente de datos específica a la que acceder.

Los comandos o instrucciones del usuario se envían a la base de datos y los resultados se devuelven al usuario. La fuente de datos puede estar ubicada en otra máquina a la que el usuario se conecte por red. A esto se denomina configuración **cliente/servidor**, con la máquina del usuario como cliente y la máquina que aloja los datos como servidor.



En el **modelo de tres capas**, los comandos se envían a una capa intermedia de servicios, la cual envía los comandos a la fuente de datos. La fuente de datos procesa los comandos y envía los resultados de vuelta a la capa intermedia, desde la que luego se le envían al usuario.



Como comentábamos anteriormente, hoy en día, hay cuatro tipos de controladores JDBC en uso:

- **Tipo 1:** puente JDBC-ODBC
- **Tipo 2:** controlador parcial de Java
- **Tipo 3:** controlador Java puro para middleware de base de datos
- **Tipo 4:** controlador Java puro para directo a base de datos

Indica si la afirmación es verdadera o falsa:

ODBC es lo mismo que JDBC, el primero para Windows y el segundo para Linux y Mac.

- a. Verdadero
- b. Falso

Conectores tipo 1 y tipo 2

Tipo 1: puente JDBC-ODBC

Los **conectores tipo 1** se denominan también **JDBC-ODBC Bridge** (puente JDBC-ODBC).

Proporcionan un puente entre el **API JDBC** y el **API ODBC**. El **driver JDBC-ODBC Bridge** traduce las llamadas JDBC a llamadas ODBC y las envía a la fuente de datos ODBC.

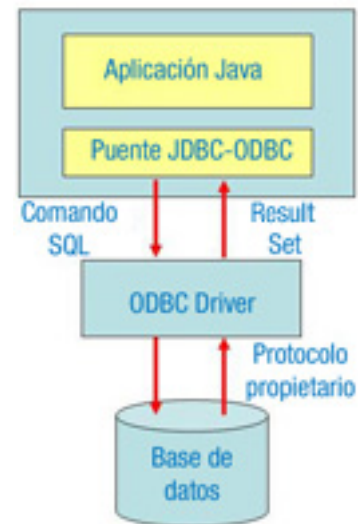
Como ventajas destacar:

- No se necesita un driver específico de cada base de datos de tipo ODBC.
- Está soportado por muchos fabricantes, por lo que tenemos acceso a muchas Bases de Datos.

Como desventajas señalar:

- Hay plataformas que no lo tienen implementado.
- El rendimiento no es óptimo ya que la llamada JDBC se realiza a través del puente hasta el conector ODBC y de ahí al interface de conectividad de la base de datos. El resultado recorre el camino inverso.
- Se tiene que registrar manualmente en el gestor de ODBC teniendo que configurar el DSN (Data Source Names, Nombres de fuentes de datos).

Este tipo de driver va incluido en el JDK.



Tipo 2: controlador parcial de Java

Los **conectores tipo 2** se conocen también como: **API nativa**

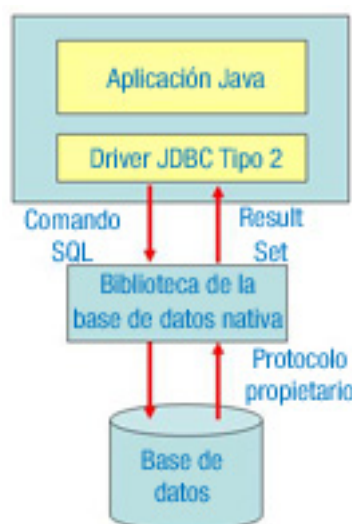
Convierten las llamadas JDBC a llamadas específicas de la base de datos para bases de datos como SQL Server, Informix, Oracle, o Sybase.

El conector **tipo 2** se comunica directamente con el servidor de bases de datos, por lo que es necesario que haya código en la máquina cliente.

Como ventaja, este conector destaca por ofrecer un rendimiento notablemente mejor que el JDBC-ODBC Bridge.

Como inconveniente, señalar que la librería de la bases de datos del vendedor necesita cargarse en cada máquina cliente. Por esta razón los drivers tipo 2 no pueden usarse para Internet.

Los drivers **Tipo 1** y **2** utilizan código nativo vía JNI, por lo que son más eficientes.



Esquema donde se ven en la parte superior la aplicación java con un driver tipo2, más abajo la librería de la base de datos nativa y más abajo la base de datos.

En estos enlaces puedes ver más información sobre JNI y sobre DSN:

http://es.wikipedia.org/wiki/Java_Native_Interface
<https://www.youtube.com/watch?v=s9rzjZ-ocg0>

Indica si la afirmación es verdadera o falsa:

El problema de los conectores tipo 2 es que no pueden utilizarse para Internet.

- a. Verdadero
- b. Falso

Conectores tipo 3 y tipo 4

Tipo 3: JDBC-Net pure Java driver

Tiene una aproximación de tres capas. Las peticiones **JDBC** a la base de datos se pasan a través de la red al servidor de la capa intermedia (**middleware**). Este servidor traduce este protocolo independiente del sistema gestor a protocolo específico del sistema gestor y se envía a la base de datos. Los resultados se mandan de vuelta al middleware y se enrutan al cliente.

Es útil para aplicaciones en Internet.

Este driver está basado en servidor, por lo que no se necesita ninguna librería de base de datos en las máquinas clientes.

Normalmente, un driver de tipo 3 proporciona soporte para balanceo de carga, funciones avanzadas de administrador de sistemas tales como auditoría, etc.

Tipo 4: Protocolo nativo.

En este caso se trata de conectores que convierten directamente las llamadas **JDBC** al protocolo de red usando por el sistema gestor de la base de datos. Esto permite una llamada directa desde la máquina cliente al servidor del sistema gestor de base de datos y es una solución excelente para acceso en intranets.

Como ventaja se tiene que no es necesaria traducción adicional o capa middleware, lo que mejora el rendimiento, siendo éste mejor que en el caso de los tipos 1 y 2.

Además, no se necesita instalar ningún software especial en el cliente o en el servidor.

Como inconveniente, de este tipo de conectores, el usuario necesita un driver diferente para cada base de datos.

Un ejemplo de este tipo de conector es **Oracle Thin**.

Esquema donde se ven en la parte superior la aplicación java con un driver tipo4 y más abajo la base de datos.



Comparación de los distintos tipo de conectores:

Tipo	Descripción	Desventajas principales
Tipo 1	JDBC-ODBC bridge	Requiere ODBC, no es portátil, obsoleto desde Java 8
Tipo 2	Driver Java que usa librerías nativas	No portátil , requiere instalación de librerías nativas
Tipo 3	Driver Java para middleware	Más complejo, requiere middleware adicional
Tipo 4	Driver Java puro para base de datos	Ninguna crítica importante. Es el estándar actual.

Conexión a una Base de Datos

Para acceder a una base de datos y así poder operar con ella, lo primero que hay que hacer es conectarse a dicha base de datos.

En Java, para establecer una conexión con una base de datos podemos utilizar el método `getConnection()` de la clase `DriverManager`. Este método recibe como parámetro la [URL de JDBC](#) que identifica a la base de datos con la que queremos realizar la conexión.

La ejecución de este método devuelve un [objeto Connection](#) que representa la conexión con la base de datos.

Cuando se presenta con una URL específica, `DriverManager` itera sobre la colección de drivers registrados hasta que uno de ellos reconoce la URL especificada. Si no se encuentra ningún conector adecuado, se lanza una [SQLException](#).

Aquí tienes un ejemplo completo de cómo conectarte a una base de datos [MySQL local](#) (p.e: `jardineria`) desde una aplicación Java usando JDBC con un [controlador Tipo 4](#):

Requisitos previos:

- Tener [MySQL instalado](#) y en ejecución en tu máquina local.
- Base de datos creada (por ejemplo, `jardineria`).
- Tener el [driver JDBC de MySQL](#) (por ejemplo: `mysql-connector-j-9.0.0.jar`) agregado al classpath del proyecto.

NOTA: La forma de agregar el conector `.jar` a nuestro proyecto dependerá del IDE que estemos utilizando (NetBeans, Eclipse, VsCode...). Por eso a continuación se pide investigar, dado el ejemplo que se expone a continuación, en tu IDE, cómo hacerlo para poder conectarte a una base de datos en local.

Como ayuda, revisa este video que te guiará en este proceso (descarga de la librería, agregar la librería al classpath etc...), el video usa como IDE Eclipse pero puedes aplicar muchas de las ideas a tu IDE.

<https://www.youtube.com/watch?v=TipyOAYGsdC>

Ejemplo:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class ConexionMySQL {
    public static void main(String[] args) {
        // Datos de conexión
        String url = "jdbc:mysql://localhost:3306/jardineria "; // Cambia el nombre de la base de datos
        String usuario = "root"; // Usuario MySQL
        String contraseña = "tu_contraseña"; // Contraseña MySQL

        // try-with-resources asegura que se cierre la conexión automáticamente
        try (Connection conexion = DriverManager.getConnection(url, usuario, contraseña)) {
            System.out.println("Conexión exitosa a la base de datos MySQL.");

            // Aquí puedes ejecutar consultas

        } catch (SQLException e) {
            System.out.println("Error al conectar con la base de datos.");
            e.printStackTrace();
        }
    }
}
```

En el enlace siguiente puedes ver cómo instalar MySQL:

<https://downloads.mysql.com/archives/community/>

Además del servidor, puedes descargarte e instalar una herramienta gráfica que permite entre otras cosas trabajar para crear tablas, editarlas, añadir datos a las tablas, etc. con MySQL. Aquí puedes ver cómo descargarlo y utilizarlo:

Instalación y uso de MySQL WorkBench:

<https://downloads.mysql.com/archives/workbench/>
<https://platzi.com/tutoriales/9873-db-sql/37668-instalacion-de-mysql-server-y-mysql-workbench-para-mac-os/>

Creación de la base de datos

Una base de datos puede crearse utilizando las herramientas proporcionadas por el fabricante de la base de datos, o por medio de sentencias SQL desde un programa Java. Pero normalmente es el administrador de la base de datos, a través de las herramientas que proporcionan el sistema gestor, el que creará la base de datos. No todos los conectores **JDBC** soportan la creación de la base de datos mediante el lenguaje de definición de datos (**DDL**). Es decir, la sentencia **CREATE DATABASE** no es parte del estándar SQL, sino que es dependiente del sistema gestor de la base de datos.

Así pues, mediante **JDBC** podemos conectarnos y manipular bases de datos: crear tablas, modificarlas, borrarlas, añadir datos en las tablas, etc. Pero la creación en sí de la base de datos la hacemos con la herramienta específica para ello.

Normalmente, cualquier sistema gestor de bases de datos incluye asistentes gráficos para crear la base de datos, sus tablas, claves, y todo lo necesario.

También, como en el caso de **MySQL**, o de **Oracle**, y la mayoría de sistemas gestores de bases de datos, se puede crear la base de datos, desde la línea de comandos de **MySQL** o de **Oracle**, con las sentencias **SQL** apropiadas.

Si tienes olvidado **MySQL**, o no lo has visto antes, deberías familiarizarte con él:

<https://www.w3schools.com/mysql/default.asp>

https://es.wikipedia.org/wiki/Clave_primaria

Hazte el Quiz MySQL y comprueba tus conocimientos

https://www.w3schools.com/mysql/mysql_quiz.asp

Drivers de conexión (Oracle y MySQL)

Driver de MySQL

MySQL proporciona controladores basados en estándares para JDBC, ODBC y .Net, lo que permite a los desarrolladores crear aplicaciones de base de datos en el idioma de su elección. Puedes descargarlos en la pagina de [MySQL](https://www.mysql.com/products/connector/) (<https://www.mysql.com/products/connector/>).

Respecto a la implementación de JDBC, [MySQL](#) proporciona conectividad para aplicaciones cliente desarrolladas en el lenguaje de programación Java con [MySQL Connector / J](#). [Connector / J](#) implementa la API de Java Database Connectivity (JDBC) , así como una serie de extensiones de valor añadido al driver. También soporta el nuevo X DevAPI.

[MySQL Connector / J](#) es un controlador JDBC Tipo 4. Existen diferentes versiones disponibles que son compatibles con las especificaciones JDBC 3.0 y JDBC 4.2. El controlador es una implementación Java pura del protocolo MySQL y no se basa en las bibliotecas cliente de MySQL.

Para los programas a gran escala que usan patrones de diseño comunes de acceso a datos, podemos usar marcos de persistencia (persistence frameworks) como [Hibernate](#) , las plantillas [JDBC de Spring](#) o el mapeo de sentencias [SQL de MyBatis](#) para reducir la cantidad de código JDBC para que pueda depurar, ajustar, proteger y mantener.

Podrás encontrar más información de los drivers JDBC disponibles en la pagina de [MySQL](#) (<https://dev.mysql.com/doc/connector-j/en/connector-j-upgrading-to-8.0.html>)

Driver de Oracle

1. [JDBC Thin Driver](#), utiliza sockets de Java para conectarse directamente a Oracle. Proporciona su propia versión TCP / IP del protocolo SQL * Net de Oracle . Debido a que es 100% Java, este controlador es independiente de la plataforma y también puede ejecutarse desde un navegador web. Los formatos de URL:

- SID: (ya no se recomienda para uso de Oracle)

```
jdbc: oracle: thin: [<user> / <password>] @ <host> [: <port>]: <SID>
```

- Servicios:

```
jdbc: oracle: thin: [<user> / <password>] @ // <host> [: <port>] / <service>
```

- TNSNames:

```
jdbc: oracle: thin: [<user> / <password>] @ <TNSName>
```

2. **JDBC OCI (Oracle Call Interfaces)** funciona a través de SQL * Net . Los controladores OCI de JDBC le permiten llamar al OCI directamente desde Java, lo que proporciona un alto grado de compatibilidad con una versión específica de Oracle. Debido a que utilizan métodos nativos, son específicos de la plataforma.

```
jdbc: oracle: oci: @myhost: 1521: orcl
```

3. **JDBC KPRB driver (default connection)**

El controlador JDBC KPRB de Oracle se utiliza principalmente para escribir procedimientos almacenados Java y JSP (JavaServer Pages). Utiliza la sesión de base de datos actual y, por lo tanto, no requiere un nombre de usuario, contraseña o URL de base de datos adicional. Se puede obtener un identificador de la conexión predeterminada o actual (controlador KPRB) llamando al método `OracleDriver.defaultConnection ()`.

Encontraras la documentación de estos tres tipos de drivers en la pagina : JDBC Oracle

<http://www.orafaq.com/wiki/JDBC>

Instalar el conector de la base de datos

MySQL

Entre nuestra aplicación [Java](#) y el Sistema Gestor de Base de Datos ([SGBD](#)), se intercala el conector [JDBC](#). Este conector es el que implementa la funcionalidad de las clases de acceso a datos y proporciona la comunicación entre el API [JDBC](#) y el [SGBD](#).

La función del conector es traducir los comandos del API JDBC al protocolo nativo del SGBD.

Vamos a utilizar el [IDE Eclipse](#) para conectar java a las bases de datos, necesitamos instalar el conector de la base de datos. Puedes seguir este tutorial, que explica la descarga del driver y la configuración necesaria para utilizarlo en un proyecto Elipse. O bien, puedes seguir el video presentación. Como verás, tan sólo consiste en descargar un archivo, descomprimirlo y desde Eclipse añadir el fichero .jar que constituye el driver que necesitamos. Es importante que compruebes el driver mas adecuado para trabajar con tu Servidor [MySQL](#).

<https://www.youtube.com/watch?v=TipyOAYGsdC>

Nota: En sistemas antiguos, para que [DriverManager](#) tuviera "registrados" los drivers, era necesario cargar la clase en la máquina virtual. Para eso es el [Class.forName\(\)](#), simplemente carga la clase con el nombre indicado.

A partir de JDK 6, los drivers JDBC 4 ya se registran automáticamente y no es necesario el [Class.forName\(\)](#), sólo que estén en el classpath de la JVM.

Ejemplo de conexión a una base de datos MySQL:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class PrimeraConexion {
    public static void main(String[] args) {
        try {
            // Establecemos la conexión con la BD
            Connection conexion = DriverManager.getConnection(
                "jdbc:mysql://localhost/ejemplo", "root", "system");
            System.out.println("conecta");
            conexion.close(); // Cerrar conexión
        } catch (SQLException cn) {
            cn.printStackTrace();    }    }    }
```

Oracle

Para **Oracle** la configuración del driver se hace de la misma forma que en **MySQL**:

Utilizaremos el driver **ojdbc8.jar** que podemos descargar en la pagina **Oracle**.

<https://www.oracle.com/database/technologies/appdev/jdbc-downloads.html>

Desde Eclipse añadimos esta librería: configuramos **Build Path**.

La cadena de conexión recomendada es la que utiliza el **ServiceName**:

```
host:port:service_name
```

Consulta si es necesario, el archivo **Tnsnames.ora** que proporciona información de las conexiones de la base de datos que estas utilizando, la ruta es:

Oracle_Home\app\oracle\product\11.2.0\server\network\ADMIN.

En este caso el **TNSNames.ora** tiene una entrada correspondiente al servicio XE que equivale a la base de Datos que se va a usar:

```
XE =  
  (DESCRIPTION =  
    (ADDRESS = (PROTOCOL = TCP)(HOST = PC)(PORT = 1521))  
    (CONNECT_DATA =  
      (SERVER = DEDICATED)  
      (SERVICE_NAME = XE))  
  )
```

La URL seria para esta base de datos:

```
String urljdbc = "jdbc:oracle:thin:@localhost:1521:XE";
```

Ejemplo de conexión a una base de datos Oracle:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class PrimeraConexion {
    public static void main(String[] args) {
        try {
            // Establecemos la conexion con la BD
            String urljdbc = "jdbc:oracle:thin:@localhost:1521:XE";
            conexion = DriverManager.getConnection(urljdbc, "ejemplo", "ejemplo");
            System.out.println("conecta");
            conexion.close(); // Cerrar conexión
        } catch (SQLException cn) {
            cn.printStackTrace();
        }
    }
}
```

JDBC conexión a una base de datos

La [API JDBC](#), aparte de algunas clases específicas, mayoritariamente está compuesto de interfaces que el controlador implementa dándoles la funcionalidad adecuada.

Para asegurar la interoperabilidad, las aplicaciones no referenciarán nunca las clases concretas de ningún controlador sino las interfaces estándares de la [API JDBC](#). Para ello, la aplicación nunca podrá instanciar directamente los objetos [JDBC](#) con una sentencia [new](#) , sino que se crearán indirectamente llamando algún método de alguna clase u objeto ya existente.

La interfaz [Connection](#) representa una conexión a la base de datos, una vía de comunicación entre la aplicación y el SGBD. Los objetos [Connection](#) mantendrán la capacidad de comunicarse con el sistema gestor mientras permanezcan abiertos. Esto es, desde que se crean hasta que se cierran utilizando el método [close](#).

El objeto [Connection](#) está totalmente vinculado a una fuente de datos, por eso en pedir la conexión hay que especificar de qué fuente se trata siguiendo el protocolo JDBC e indicando la url de los datos, y en su caso el [usuario](#) y [password](#) . Se obtiene utilizando el método [getConnection\(\)](#) de la clase [DriverManager](#). Este método recibe como parámetro la URL de JDBC que identifica a la base de datos con la que queremos realizar la conexión:

- [getConnection\(String url\)](#)
- [getConnection\(String url, Properties info\)](#)
- [getConnection\(String URL, String user, String password\);](#)

Cuando se presenta con una URL específica, [DriverManager](#) itera sobre la colección de drivers registrados hasta que uno de ellos reconoce la URL especificada. Si no se encuentra ningún conector adecuado, se lanza una [SQLException](#).

La forma más sencilla de URL JDBC es una lista de tres valores separados mediante dos puntos. El primer valor de la lista representa el [protocolo](#), que es siempre [jdbc](#) para los URL JDBC. El segundo valor es el [subprotocolo](#) . El tercer valor es el nombre de sistema para establecer la conexión con un sistema específico, dependerá del tipo de controlador utilizado, del [host](#) donde se aloje el SGBD, del [puerto](#) que este use para escuchar las peticiones y del [nombre de la base de datos](#) o esquema con el que queremos trabajar.

```
jdbc:mysql://localhost/ejemplo
```

Existen dos valores especiales que pueden utilizarse para conectarse con la base de datos local. Son [*LOCAL](#) y [localhost](#) (ambos son sensibles a mayúsculas y minúsculas).

- Para conectarnos a una base de [datos Oracle 11XE](#):

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;
public class PrimeraConexion {
    public static void main(String[] args) {
        Connection conexion;
        try {
            //url oracle con formato jdbc:oracle:thin:@<hostname>:<port>:<servicename>
            //url con formato jdbc:oracle:thin:<usuario>@<hostname>:<port>:<servicename>
            String urljdbc = "jdbc:oracle:thin:ejemplo/ejemplo@localhost:1521:XE";
            //establecemos la conexión utilizando el método getConnection
            //pasamos url, nombre del usuario y contraseña
            conexion = DriverManager.getConnection(urljdbc, "ejemplo", "ejemplo");
            System.out.println("conecta");
            conexion.close();
        } catch (SQLException e) {
            e.printStackTrace();
        }
    }
}
```

- Para establecer la conexión con la **BD MySQL**, únicamente hay que utilizar el driver JDBC correspondiente y configurar el proyecto para añadir la librería **mysql-connector-java-x.x.x-bin.jar** .

Para obtener mas información de los drivers de mysql disponibles y de la sintaxis de url correspondiente a cada uno de ellos, visita.

<https://dev.mysql.com/doc/index-connectors.html>

La url para una BD local es: ***"jdbc:mysql://localhost/base datos"***

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class PrimeraConexion {
    public static void main(String[] args) throws SQLException {
        // Establecemos la conexion con la BD
        Connection conexion = DriverManager.getConnection("jdbc:mysql://localhost/ejemplo", "root", "system");
        // Cerrar conexión
        conexion.close();
    }
}
```

A este código le falta la gestión de errores!!

Realización de consultas

Para operar con una base de datos, ejecutando las consultas necesarias, nuestra aplicación deberá:

- Cargar el driver necesario para comprender el protocolo que usa la base de datos en cuestión.
- Establecer una conexión con la base de datos.
- Enviar consultas SQL y procesar el resultado.
- Liberar los recursos al terminar.
- Gestionar los errores que se puedan producir.

Para hacer consultas en la base de datos tendremos que crear un objeto de tipo [Statement](#), ejecutar la consulta con el método [executeQuery\(sql\)](#) y recorrer el [ResultSet](#) que devuelve este método.

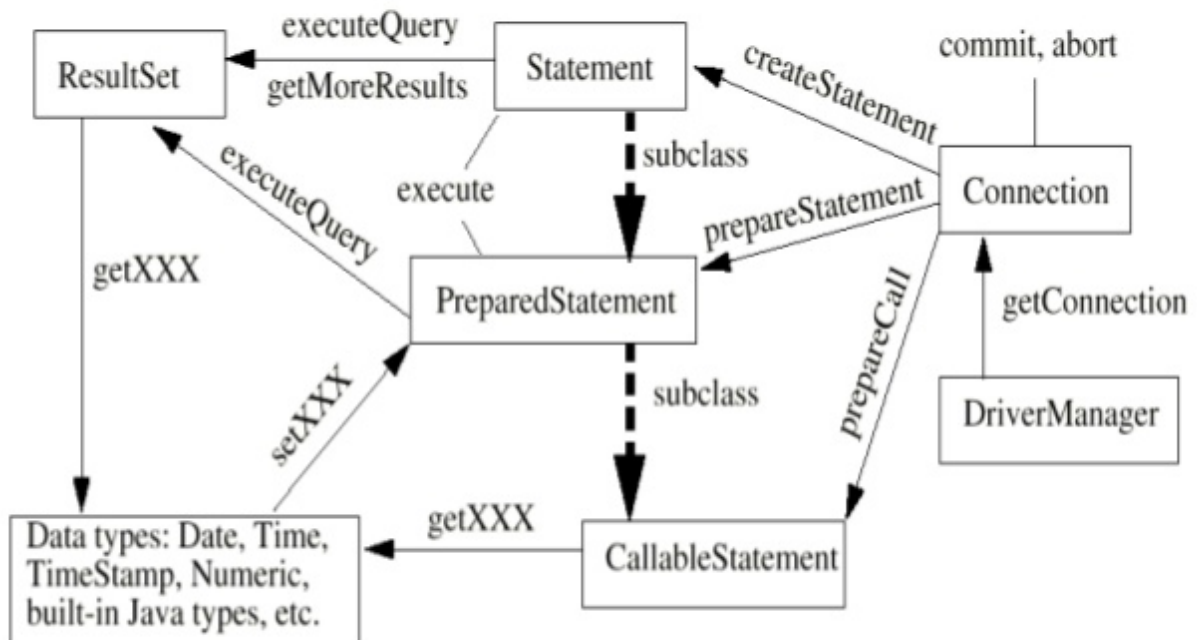
Ejemplo 1:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class PrimeraConsulta {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost/ejemplo";
        String usuario = "root";
        String contraseña = "system";
        String sql = "SELECT * FROM dept";

        try (
            Connection conexion = DriverManager.getConnection(url, usuario, contraseña);
            Statement sentencia = conexion.createStatement();
            ResultSet resul = sentencia.executeQuery(sql)
        ) {
            while (resul.next()) {
                System.out.printf("%d, %s, %s\n", resul.getInt(1), resul.getString(2), resul.getString(3));
            }
        } catch (SQLException e) {
            System.out.println("Error al acceder a la base de datos:");
            e.printStackTrace();
        }
    }
}
```

JDBC Class Diagram



www.SunilOS.com

35

El API JDBC distingue dos tipos de sentencias SQL:

- Consultas: [SELECT](#) devuelven de ninguna o varias filas.
- Actualizaciones: [INSERT](#), [UPDATE](#), [DELETE](#), sentencias DDL.

SELECT

Veamos en primer lugar consultas que devuelven datos (SELECT)

Los datos se devuelven utilizando un objeto [ResultSet](#). Se accede a los datos a través de un cursor. Este cursor es un puntero que apunta a una fila de datos en el objeto [ResultSet](#). Inicialmente, el cursor se coloca antes de la primera fila. Accedemos a todas las filas de una a una llamando el método [next](#), ya que cada invocación [next](#) avanzará a la siguiente fila.

Ejemplo 2:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConsultaSencilla {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521:XE"; // Asegúrate de que este formato sea correcto
        String usuario = "ejemplo";
        String contraseña = "ejemplo";

        String sql = "SELECT empno, ename, job FROM emp ORDER BY empno";

        try (
            Connection conexion = DriverManager.getConnection(url, usuario, contraseña);
            Statement smt = conexion.createStatement();
            ResultSet rset = smt.executeQuery(sql)
        ) {
            while (rset.next()) {
                System.out.printf(
                    "Empleado número %s, Nombre: %s, Oficio: %s%n",
                    rset.getString(1),
                    rset.getString(2),
                    rset.getString(3)
                );
            }
        }
    }
}
```

```
    } catch (SQLException e) {  
        System.out.println("Error al acceder a la base de datos Oracle:");  
        e.printStackTrace();  
    }  
}  
}
```

UPDATE, DELETE, INSERT

Veamos como ejecutar sentencias DML (UPDATE, DELETE, INSERT)

Pese a que se trata de sentencias muy dispares, desde el punto de vista de la comunicación con el SGBD se comportan de manera muy similar, siguiendo el siguiente patrón:

- A partir de una conexión activa, instanciamos un objeto [Statement](#).
- Ejecutamos la sentencia SQL, método [executeUpdate](#).
- Cerramos el objeto [Statement](#) instanciado.
- Cerramos la conexión activa.

El método [executeUpdate](#) devuelve un número entero que representa el número de filas afectadas por la instrucción SQL.

Veamos el ejemplo siguiente, insertamos una línea y luego borramos todas las filas de la tabla Jobs.

```
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;  
import java.sql.Statement;  
  
public class Actualizacion {  
    public static void main(String[] args) {  
        String url = "jdbc:oracle:thin:@localhost:1521:XE"; // Formato correcto de conexión Oracle  
        String usuario = "scott";  
        String contraseña = "tiger";  
  
        try (  
            Connection conexion = DriverManager.getConnection(url, usuario, contraseña);  
            Statement stm = conexion.createStatement()  
        ) {  
            // Insertar un nuevo registro en la tabla JOBS
```

```
int filasInsertadas = stm.executeUpdate(
    "INSERT INTO jobs (job_id, job_title, min_salary, max_salary) " +
    "VALUES ('ID_TEACH', 'PROFESOR IT', 2500, 5000)"
);
System.out.println("Filas insertadas: " + filasInsertadas);

// Eliminar todos los registros de la tabla JOBS
int filasBorradas = stm.executeUpdate("DELETE FROM jobs");
System.out.println("Filas eliminadas: " + filasBorradas);

} catch (SQLException e) {
    System.out.println("Error al ejecutar actualizaciones en la base de datos Oracle:");
    e.printStackTrace();
}
}
```

Ejecución de sentencias DML

Las sentencias **DML** (Lenguaje de manipulación de datos), incluyen fundamentalmente tres tipos de operaciones inserciones (**INSERT**), borrados (**DELETE**) y modificaciones (**UPDATE**).

El funcionamiento de todas ellas en la conexión es bastante similar. A continuación se muestra un ejemplo de borrado en una conexión **Oracle**:

```
import java.sql.*;

public class DMLDeletePrep {

    public static void main(String[] args) {

        if (args.length < 1) {

            System.out.println("Debes proporcionar el número de departamento como argumento.");
            return;
        }

        String deptno = args[0];
        String url = "jdbc:oracle:thin:@localhost:1521:XE"; // Cambia si usas otra instancia
        String usuario = "scott";
        String contraseña = "tiger";
        String sql = "DELETE FROM dept WHERE DEPTNO = ?";

        // try-with-resources para manejo automático
        try (
            Connection conexion = DriverManager.getConnection(url, usuario, contraseña);
            PreparedStatement sentencia = conexion.prepareStatement(sql)
        ) {

            sentencia.setInt(1, Integer.parseInt(deptno));
            int filas = sentencia.executeUpdate();
            System.out.println("Filas afectadas: " + filas);

        } catch (SQLException e) {

            System.out.println("Error de SQL:");
            e.printStackTrace();

        } catch (NumberFormatException e) {

            System.out.println("El argumento proporcionado no es un número válido.");

        }

    }

}
```

Ejecución de Procedimientos Almacenados

Un **procedimiento almacenado** es un procedimiento o subprograma que está almacenado en la base de datos.

Muchos sistemas gestores de bases de datos los soportan, por ejemplo: **MySQL**, **Oracle**, etc.

Además, estos procedimientos suelen ser de dos clases:

- **Procedimientos** almacenados.
- **Funciones**, las cuales devuelven un valor que se puede emplear en otras sentencias SQL.

Un procedimiento almacenado típico tiene:

- Un **nombre**.
- Una lista de **parámetros**.
- Unas sentencias **SQL**.

Veamos un ejemplo de sentencia para crear un procedimiento almacenado sencillo para **MySQL**, aunque sería similar en otros sistemas gestores:

Imagen del código de un procedimiento almacenado.

```
CREATE PROCEDURE procediprueba          /* nombre del procedimiento */
(IN par1 INTEGER)                       /* parámetros */
BEGIN                                  /* Inicio del bloque */
    DECLARE var1 CHAR(13);              /* Declarar variables */
    IF par1 = 24 THEN                   /* Inicio del IF */
        SET var1 = 'perro rabioso';    /* asignar valor a la variable */
    ELSE
        SET var1 = 'gato persa';        /* asignar valor a la variable */
    END IF;                             /* Fin del IF */
    INSERT INTO Animales VALUES (var1); /* insertar registro */
END                                     /* fin de bloque */
```

Como se ve en los comentarios, este procedimiento admite un parámetro, llamado par1. También se declara una variable a la que llamamos var1 y es de tipo carácter y longitud 13. Si el valor que le llega de parámetro es igual a 24, entonces se asigna a la variable var1, la cadena 'perro rabioso' y en caso contrario se le asignará la cadena: 'gato persa'. Finalmente, se inserta en la tabla "Animales" el valor que se asignó a la variable var1.

Puedes familiarizarte con los comandos que puedes necesitar para realizar procedimientos almacenados y funciones, aquí:

<https://dev.mysql.com/doc/refman/8.4/en/create-procedure.html>

<https://dev.mysql.com/doc/connectors/en/connector-net-tutorials-stored-procedures.html>

Ejecución de procedimientos almacenados en la base de datos MySQL

A continuación, vamos a realizar un procedimiento almacenado en MySQL, que simplemente insertará datos en la tabla clientes. Desde el programa Java que realizamos, llamaremos para ejecutar a ese procedimiento almacenado. Por tanto, ¿cuál sería la secuencia que seguiríamos para realizar esto?

- ✓ Si no tenemos creada la tabla de clientes, la creamos. Por simplicidad, en este ejemplo, trabajamos sobre la base de datos que viene por defecto en MySQL, el esquema denominado: test. Para crear la tabla de clientes, el script correspondiente es:
 - Crear tabla en MySQL. (1 KB)

```
CREATE TABLE clientes (  
    Cod_Cliente INTEGER NOT NULL DEFAULT 0,  
    Nombre VARCHAR(50),  
    Telefono VARCHAR(40),  
    PRIMARY KEY (Cod_Cliente)  
);
```

- ✓ Creamos el procedimiento almacenado en la base de datos. Sería tan fácil como lo que ves en el siguiente enlace:
 - DDL rutina insertar cliente

```
DELIMITER $$  
CREATE PROCEDURE insertaCliente (IN Cod_Cliente INTEGER, IN Nombre VARCHAR(50), IN Telefono VARCHAR(40) )  
BEGIN  
    INSERT INTO clientes VALUES (Cod_Cliente, Nombre, Telefono);  
END$$
```

- ✓ Crear la clase Java para desde aquí, llamar al procedimiento almacenado:
 - Llamar al procedimiento almacenado desde un fichero .java.

```
package conexmysql;  
  
import java.sql.CallableStatement;  
import java.sql.Connection;  
import java.sql.DriverManager;  
import java.sql.SQLException;
```

```

public class InserConProcAlma {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost/test";
        String usuario = "root";
        String contraseña = "admin";

        // Sintaxis del procedimiento almacenado
        String sql = "{ CALL insertaCliente(?, ?, ?) }";
        // { CALL ... } Es una convención JDBC para indicar que se está llamando a un procedim. almacenado

        try (
            Connection con = DriverManager.getConnection(url, usuario, contraseña);
            CallableStatement stmt = con.prepareCall(sql)
        ) {
            // Cargar parámetros
            stmt.setInt("Cod_Cliente", 765);
            stmt.setString("Nombre", "Antonio Pérez");
            stmt.setString("Telefono", "950121314");

            // NOTA: Si tuviéramos un parámetro de salida p.e: Result , se utilizaría esto:
            // stmt.registerOutParameter(Result, Types.VARCHAR);
            // y después del stmt.execute(); se recuperaría el valor del parámetro de salida así:
            // String resultado = stmt.getString(Result);

            // Ejecutar
            stmt.execute();
            System.out.println("Procedimiento almacenado ejecutado con éxito.");

        } catch (SQLException e) {
            System.err.println("Error SQL: " + e.getMessage());
            e.printStackTrace();
        }
    }
}

```

Si hemos definido la tabla correctamente, con su clave primaria, y ejecutamos el programa, intentando insertar una fila igual que otra insertada, o sea, con la misma clave primaria, obtendremos un mensaje al capturar la excepción de este tipo:

```
SQL Exception:  
com.mysql.jdbc.exceptions.jdbc4.MySQLIntegrityConstraintViolationException:  
Duplicate entry '765' for key 'PRIMARY'
```

Si quieres ver ejemplos JDBC para diversas finalidades: listar datos de una base de datos, llamar a funciones en Oracle, etc.

<https://users.dcc.uchile.cl/~lmateu/CC60H/Trabajos/jfernand/>

Ejecución de procedimientos almacenados en la base de datos Oracle

Tomando un procedimiento de almacenamiento tal que:

```
CREATE OR REPLACE PROCEDURE remove_emp (employee_id NUMBER) AS
    tot_emps NUMBER;
BEGIN
    DELETE FROM employees
    WHERE employees.employee_id = remove_emp.employee_id;
    tot_emps := tot_emps - 1;
END;
/
```

Para ejecutar el procedimiento de almacenamiento de arriba:

```
import java.sql.CallableStatement;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

public class EjemploProcAlmacenado1 {
    public static void main(String[] args) {
        Connection cn = null;
        try {
            // Carga el driver de oracle
            DriverManager.registerDriver(new oracle.jdbc.driver.OracleDriver());
            // Conecta con la base de datos XE con el usuario scott y la contraseña tiger
            cn = DriverManager.getConnection("jdbc:oracle:thin:@localhost:1521:XE", "scott", "tiger");

            // Llamada al procedimiento almacenado
            CallableStatement cst = cn.prepareCall("{call remove_emp (104)}");
            // Ejecuta el procedimiento almacenado
            cst.execute();
            System.out.println("borrado");
            cst.close();
        } catch (SQLException ex) {
            System.out.println("Error: " + ex.getMessage());
        } finally {
```

```
        try {  
            cn.close();  
        } catch (SQLException ex) {  
            System.out.println("Error: " + ex.getMessage());  
        }  
    }  
}  
}
```

En el código anterior:

Las llamadas a los procedimientos almacenados al igual que las [PreparedStatements](#) y las consultas simples se hacen sobre la [conexión](#), en este caso con el método [prepareCall\(\)](#) que nos devuelve un [CallableStatement](#).

La llamada al procedimiento almacenado además de ir entre [comillas](#) por ser un string tiene que ir también entre [llaves](#) y tiene el siguiente formato [“{call nombre_procedimiento\(params\)}”](#), dando los parámetros reales espera el procedimiento almacenado.

El procedimiento se ejecuta cuando llamamos al método [execute](#), y como es lógico en el momento en el que se ejecute tienen que estar definidos todos los parámetros tanto de entrada como de salida.

Sentencias Preparadas

Muy usadas y muy útiles!!. A veces es útil usar un objeto `PreparedStatement` para enviar sentencias de SQL a la base de datos. Este tipo especial de declaración se deriva de la clase más general `Statement`.

Una `PreparedStatement` es una sentencia SQL de base de datos precompilada. Al estar precompilada, su ejecución será más rápida que una SQL normal, por lo que es adecuada cuando vamos a ejecutar la misma sentencia SQL (con distintos valores) muchas veces.



¿Cómo se usa?

Simplemente ponemos interrogantes donde irán los valores concretos que vayamos a insertar.

Cada interrogante se identifica luego con un número, de forma que el primer interrogante que aparece es el 1, el segundo el 2, etc.

Luego, con los métodos `set()` correspondientes rellenamos esos valores.

El primer parámetro es el número del interrogante y el segundo el valor que queremos insertar. Finalmente llamamos a `executeUpdate()`. Luego, con la misma `PreparedStatement`, ponemos otros valores volviendo a llamar a los métodos `set()` y volvemos a llamar a `executeUpdate()`

Fíjate que a la hora de poner los interrogantes, no nos hemos preocupado de poner comillas en los valores de texto ni conversiones de ningún tipo. Las llamadas a `setInt()`, `setString()`, ya hacen todas las conversiones adecuadas para nosotros.

Ejemplo 1: El siguiente ejemplo utiliza un único objeto `PreparedStatement`, dándole valores a los parámetros:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class ActualizaSentenciaPreparada {
    public static void main(String[] args) {
        String url = "jdbc:oracle:thin:@localhost:1521:XE";
        String usuario = "scott";
        String contraseña = "tiger";
        String sql = "UPDATE EMP SET SAL = ? WHERE empno = ?";

        try {
            Connection conexion = DriverManager.getConnection(url, usuario, contraseña);
            PreparedStatement pstmt = conexion.prepareStatement(sql)

            pstmt.setInt(1, 1500); // Nuevo salario 1500
            pstmt.setInt(2, 7844); // Número de empleado 7844

            int filasAfectadas = pstmt.executeUpdate();
            System.out.println("Filas actualizadas: " + filasAfectadas);
        } catch (SQLException e) {
            System.err.println("Error al actualizar:");
            e.printStackTrace();
        }
    }
}
```

Los métodos que permiten pasar parámetros a la sentencia empiezan por set. A continuación se muestran estos métodos:

```
setBigDecimal(int parameterIndex, BigDecimal x)
setBinaryStream(int parameterIndex, InputStream x)
setDate(int parameterIndex, Date x)
setDouble(int parameterIndex, double x)
setInt(int parameterIndex, int x)
setString(int parameterIndex, String x)
etc...
```

Ejemplo 2: En el siguiente ejemplo veremos cómo recuperar el ID de un departamento que daremos de alta para luego insertar empleados que trabajan en ese departamento. Pasos:

1. **Insertar un nuevo departamento** en una tabla con un ID auto-incremental.
2. **Obtener ese ID generado** automáticamente (el ID del nuevo departamento).
3. **Insertar un empleado** usando ese ID como clave foránea.

Tenemos estas tablas:

```
CREATE TABLE departamentos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL
);

CREATE TABLE empleados (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    departamento_id INT,
    FOREIGN KEY (departamento_id) REFERENCES departamentos(id)
);
```

El código Java correspondiente será:

```
import java.sql.*;

public class InsertarEmpleadoConDepartamento {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/ejemplo";
        String usuario = "root";
        String password = "system";

        try (Connection conexion = DriverManager.getConnection(url, usuario, password)) {
            conexion.setAutoCommit(false); // Transacción manual

            try {
                // 1. Insertar el nuevo departamento
                String sqlDepto = "INSERT INTO departamentos (nombre) VALUES (?)";
                try (PreparedStatement psDepto = conexion.prepareStatement(sqlDepto, Statement.RETURN_G
ENERATED_KEYS)) {
                    psDepto.setString(1, "Desarrollo");
                    psDepto.executeUpdate();

                    // Obtener el ID generado
                    int nuevoIdDepartamento = -1;
                    try (ResultSet rs = psDepto.getGeneratedKeys()) {
                        if (rs.next()) {
                            nuevoIdDepartamento = rs.getInt(1);
                            System.out.println("Nuevo ID de departamento: " + nuevoIdDepartamento);
                        }
                    }
                }

                // 2. Insertar empleado usando el ID del departamento recién creado
                String sqlEmpleado = "INSERT INTO empleados (nombre, departamento_id) VALUES (?, ?)";
                try (PreparedStatement psEmpleado = conexion.prepareStatement(sqlEmpleado)) {
                    psEmpleado.setString(1, "Juan Pérez");
                    psEmpleado.setInt(2, nuevoIdDepartamento);
                    psEmpleado.executeUpdate();
                    System.out.println("Empleado insertado.");
                }
            }
        }
    }
}
```

```
        }  
    }  
  
    conexion.commit(); // Todo correcto  
} catch (SQLException e) {  
    conexion.rollback(); // Si algo falla  
    System.err.println("Transacción cancelada. Se hizo rollback.");  
    e.printStackTrace();  
}  
  
} catch (SQLException e) {  
    e.printStackTrace();  
}  
}  
}
```

- **Statement.RETURN_GENERATED_KEYS** permite recuperar el ID autogenerado.
- El método **getGeneratedKeys()** te da acceso al ID insertado.
- Todo está envuelto en una transacción (commit y rollback) para garantizar consistencia ACID, lo veremos más adelante.

Consulta este video para ver “Cómo son las Consultas Preparadas”:

<https://www.youtube.com/watch?v=OOWg6m1B7uo>

Captura de errores y liberación de recursos

Liberación de recursos

Debemos tener en cuenta siempre que las conexiones con una base de datos consumen muchos recursos en el sistema gestor, y por lo tanto en el sistema informático en general. Por ello, conviene cerrarlas con el método `close` (de la clase `Connection`) siempre que vayan a dejar de ser utilizadas, en lugar de esperar a que el garbage collector de Java las elimine.

También conviene cerrar la ejecución de procedimientos almacenados `CallableStatement`, las consultas (`Statement` y `PreparedStatement`) y los resultados (`ResultSet`) para liberar los recursos.

Gestión de errores

El manejo de excepciones permite manejar condiciones excepcionales tales como errores definidos por el programa de una manera controlada.

Cuando se produce una condición de excepción, se lanza una excepción. La ejecución del programa actual se detiene y el control se redirige a la cláusula de captura (`catch`) más cercana. Si no existe, la ejecución del programa finaliza.

El manejo de excepciones de JDBC es muy similar al manejo de excepciones de Java, pero para JDBC, la excepción más común con la que tratará es `java.sql.SQLException`.

Se puede producir una `SQLException` tanto en el controlador como en la base de datos. Cuando se produce una excepción de este tipo, un objeto de tipo `SQLException` se pasará a la cláusula `catch`.

El objeto `SQLException` pasado tiene los siguientes métodos disponibles para recuperar información adicional sobre la excepción:

```
getErrorCode ()  
getMessage ()  
printStackTrace ()  
etc.
```

Si quieres más información sobre las excepciones de java

<https://elvex.ugr.es/decsai/java/pdf/B2-excepciones.pdf>

Try-with-resources

En Java, la liberación de recursos es crucial para evitar fugas de memoria y garantizar que los recursos externos, como archivos, conexiones de bases de datos o sockets, se cierren correctamente. Desde Java 7, se introdujo la sintaxis `try-with-resources`, que facilita la gestión automática de los recursos, asegurando que se cierren de manera adecuada, incluso si se produce una excepción.

¿Qué es `try-with-resources`?

El bloque `try-with-resources` es una mejora sobre el `try-catch-finally`, que permite abrir y cerrar automáticamente recursos que implementen la interfaz `AutoCloseable`. Cuando un recurso se usa en un bloque `try-with-resources`, **no es necesario cerrarlo manualmente en un bloque `finally`**, ya que el cierre se maneja automáticamente al final del bloque.

Sintaxis básica de `try-with-resources`:

```
try (Recurso recurso = new Recurso()) {  
    // Uso del recurso  
} catch (Exception e) {  
    // Manejo de excepciones  
}
```

- **Recurso:** Debe ser una clase que implemente la interfaz `AutoCloseable` o su subinterfaz `Closeable`. Los recursos más comunes que implementan esta interfaz son archivos, conexiones de bases de datos, y flujos de entrada/salida (IO).

Ventajas de `try-with-resources`

- **Código más limpio y legible:** No es necesario implementar un bloque `finally` para cerrar los recursos.
- **Cierre automático de recursos:** Los recursos se cierran automáticamente, lo que reduce el riesgo de fugas de memoria.
- **Manejo de excepciones:** Si ocurre una excepción durante el uso de un recurso, se manejará normalmente en el bloque `catch`. El recurso se cerrará incluso si ocurre una excepción.

Interfaz `AutoCloseable`

Para que un recurso pueda ser utilizado en un bloque `try-with-resources`, debe implementar la interfaz `AutoCloseable` o `Closeable` (esta última es una subinterfaz de `AutoCloseable`).

`AutoCloseable`: Se usa generalmente para cualquier recurso que deba cerrarse, ya sea una conexión de base de datos, un archivo, un flujo de datos, etc.

`Closeable`: Es una subinterfaz de `AutoCloseable` y se usa específicamente para flujos de entrada/salida (IO).

Ejemplo de try-with-resources:

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class Main {

    // URL de la base de datos, usuario y contraseña
    private static final String DB_URL = "jdbc:mysql://localhost:3306/mi_base_datos";
    private static final String USER = "root";
    private static final String PASSWORD = "mi_password";

    public static void main(String[] args) {
        String query = "SELECT id, nombre, fecha_nacimiento FROM empleados WHERE id = ?";

        try (
            // 1. Crear la conexión a la base de datos (se cierra automáticamente)
            Connection connection = DriverManager.getConnection(DB_URL, USER, PASSWORD);
            // 2. Crear el PreparedStatement (se cierra automáticamente)
            PreparedStatement preparedStatement = connection.prepareStatement(query)
        ) {
            // Establecer el parámetro de la consulta
            preparedStatement.setInt(1, 11); // Suponemos que queremos obtener el empleado con id = 11

            // 3. Ejecutar la consulta y obtener el resultado (se cierra automáticamente)
            try (ResultSet resultSet = preparedStatement.executeQuery()) {

                // Si se encuentra el registro
                if (resultSet.next()) {
                    // Obtener los datos de las columnas
                    int id = resultSet.getInt("id");
                    String nombre = resultSet.getString("nombre");
                    String fechaNacimiento = resultSet.getString("fecha_nacimiento");
                }
            }
        }
    }
}
```

```
        // Mostrar los datos obtenidos
        System.out.println("ID: " + id);
        System.out.println("Nombre: " + nombre);
        System.out.println("Fecha de nacimiento: " + fechaNacimiento);
    } else {
        System.out.println("No se encontró el empleado con el ID proporcionado.");
    }
}
} catch (SQLException e) {
    e.printStackTrace(); // Manejar cualquier excepción relacionada con la base de datos
}
}
```

Transacciones

Cuando tenemos una serie de consultas SQL que deben ejecutarse en conjunto, con el uso de transacciones podemos asegurarnos de que nunca nos quedaremos a medio camino de su ejecución.

Las transacciones tienen la característica de poder “**deshacer**” los cambios efectuados en las tablas, de una transacción dada, si no se han podido realizar todas las operaciones que forman parte de dicha transacción.

Por eso, las **bases de datos** que soportan transacciones son mucho más **seguras** y **fáciles** de recuperar si se produce algún fallo en el servidor que almacena la base de datos, ya que las consultas se ejecutan o no en su totalidad.

Al ejecutar una transacción, el motor de base de datos garantiza:

- ✓ Atomicidad (**A**tomicity)
- ✓ Consistencia (**C**onsistency)
- ✓ Aislamiento (**I**solation)
- ✓ Durabilidad (**D**urability)

(**ACID**) de la transacción (o conjunto de comandos) que se utilice.



Resumen de ACID:

Propiedad	Descripción
Atomicidad	Una transacción es un todo: o se realiza completamente o no se realiza en absoluto.
Consistencia	La transacción deja la base de datos en un estado válido y consistente antes y después de la transacción.
Aislamiento	Las transacciones concurrentes no deben interferir entre sí. Cada transacción ocurre como si fuera la única en ejecución.
Durabilidad	Una vez que una transacción se confirma, sus resultados persisten incluso si hay un fallo del sistema.

El ejemplo típico que se pone para hacer más clara la necesidad de transacciones en algunos casos es el de una transacción bancaria. Por ejemplo, si una cantidad de dinero es transferida de la cuenta de Antonio a la cuenta de Pedro, se necesitarían dos consultas:

En la cuenta de Antonio para quitar de su cuenta ese dinero:

```
UPDATE cuentas SET saldo = saldo - cantidad WHERE cliente = "Antonio";
```

En la cuenta de Pedro para añadir ese dinero a su cuenta:

```
UPDATE cuentas SET saldo = saldo + cantidad WHERE cliente = "Pedro";
```

Pero, ¿qué ocurre si por algún imprevisto (un apagón de luz, etc.), el sistema “cae” después de que se ejecute la primera consulta, y antes de que se ejecute la segunda? Antonio tendrá una cantidad de dinero menos en su cuenta y creerá que ha realizado la transferencia. Pedro, sin embargo, creerá que todavía no le han realizado la transferencia.

Respecto a las transacciones, señala la respuesta correcta:

En caso de una transacción con tres operaciones, no se podrá deshacer en ningún caso.

- a. Una transacción permite recuperar los datos si se produce un fallo, aportando por tanto más seguridad en la realización de operaciones en la base de datos.
- b. Una transacción se hará sólo si las operaciones involucradas operan con dinero.
- c. Ninguna es correcta.

De manera más formal el control de la transacción es realizado por el objeto de la conexión. Cuando se crea una conexión, por defecto esta en el modo activado. Esto significa que cada operación **DML** (**INSERT**, **UPDATE**, **DELETE**) es tratada como transacción por sí misma que se valida automáticamente en cuanto se ejecute.

A veces necesitamos agrupar varias sentencias **SQL** en una única transacción, para ello:

- ✓ Modificamos el autocommit antes de ejecutar las consultas que deban estar en la misma transacción: `conexion.setAutoCommit(false)`
- ✓ Ejecutamos las sentencias
- ✓ Finalizaremos de forma manual la transacción. `conexion.Commit`.

Ejemplo:

Supongamos que tienes una base de datos con una tabla cuentas: existen dos cuentas y deseas transferir dinero entre dos cuentas bancarias. La operación de transferencia es una transacción porque implica dos operaciones SQL (restar de una cuenta y sumar a otra), y ambas deben ejecutarse correctamente o revertirse juntas, las actualizaciones se tratan como una única transacción de forma

que si se ha podido completar todas las actualizaciones, se validan y en caso de que falle alguna, el gestor de errores deshace los cambios (connection.rollback())

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class TransferenciaBancaria {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/banco";
        String usuario = "root";
        String contraseña = "password";

        String sqlRetirar = "UPDATE cuentas SET saldo = saldo - ? WHERE id_cuenta = ?";
        String sqlDepositar = "UPDATE cuentas SET saldo = saldo + ? WHERE id_cuenta = ?";
        int montoTransferencia = 200;

        try (Connection connection = DriverManager.getConnection(url, usuario, contraseña)) {
            connection.setAutoCommit(false); // Iniciar transacción

            try (
                PreparedStatement psRetirar = connection.prepareStatement(sqlRetirar);
                PreparedStatement psDepositar = connection.prepareStatement(sqlDepositar)
            ) {
                // Retirar de la cuenta 9001
                psRetirar.setInt(1, montoTransferencia);
                psRetirar.setInt(2, 9001);
                psRetirar.executeUpdate();

                // Depositar en cuenta 9002
                psDepositar.setInt(1, montoTransferencia);
                psDepositar.setInt(2, 9002);
                psDepositar.executeUpdate();

                // Confirmar la transacción
                connection.commit();
            }
        }
    }
}
```

```
        System.out.println("Transferencia realizada con éxito.");
    } catch (SQLException e) {
        // Si ocurre error, deshacer la transacción
        System.err.println("Error durante la transferencia. Realizando rollback...");
        connection.rollback();
        e.printStackTrace();
    } finally {
        connection.setAutoCommit(true); // Restaurar autocommit
    }

} catch (SQLException e) {
    System.err.println("Error al conectar con la base de datos.");
    e.printStackTrace();
}

}

}
```

Commit y Rollback

Una transacción tiene dos finales posibles, **COMMIT** o **ROLLBACK**. Si se finaliza correctamente y sin problemas se hará con **COMMIT**, con lo que los cambios se realizan en la base de datos, y si por alguna razón hay un fallo, se deshacen los cambios efectuados hasta ese momento, con la ejecución de **ROLLBACK**.

Por defecto, al menos en **MySQL** o con **Oracle**, en una conexión trabajamos en modo **autocommit** con valor **true**. Eso significa que cada consulta es una transacción en la base de datos. Por tanto, si queremos definir una transacción de varias operaciones, estableceremos el modo **autocommit** a **false** con el método **setAutoCommit** de la clase **Connection**. En modo no autocommit las transacciones quedan definidas por las ejecuciones de los métodos **commit** y **rollback**. Una transacción abarca desde el último commit o rollback hasta el siguiente commit. Los métodos **commit** o **rollback** forman parte de la clase **Connection**.

En la siguiente porción de código de un procedimiento almacenado, puedes ver un ejemplo sencillo de cómo se puede utilizar commit y rollback: tras las operaciones se realiza el commit, y si ocurre una excepción, al capturarla realizaríamos el rollback.

Supongamos que tenemos este procedimiento almacenado:

```
DELIMITER $$

CREATE PROCEDURE transferirFondos(IN cuentaOrigen INT, IN cuentaDestino INT, IN monto DECIMAL(10,2))
BEGIN
    UPDATE cuentas SET saldo = saldo - monto WHERE id = cuentaOrigen;
    UPDATE cuentas SET saldo = saldo + monto WHERE id = cuentaDestino;
END $$

DELIMITER ;
```

Ahora implementamos en Java el código:

```
import java.sql.*;

public class TransferenciaBancaria {
    public static void main(String[] args) {
        String url = "jdbc:mysql://localhost:3306/ejemplo";
        String usuario = "root";
        String password = "system";
```



```
try (Connection conexion = DriverManager.getConnection(url, usuario, password)) {
    conexion.setAutoCommit(false); // Desactiva autocommit para manejar transacciones manualmente

    try (CallableStatement cs = conexion.prepareCall("{call transferirFondos(?, ?, ?)}")) {
        cs.setInt(1, 1);      // ID de cuenta origen
        cs.setInt(2, 2);      // ID de cuenta destino
        cs.setBigDecimal(3, new java.math.BigDecimal("100.00")); // Monto

        cs.execute();
        conexion.commit();    // Si todo sale bien, se hace commit
        System.out.println("Transferencia realizada exitosamente.");
    } catch (SQLException ex) {
        conexion.rollback();  // En caso de error, se revierte todo
        System.err.println("Error en la transferencia. Se hizo rollback.");
        ex.printStackTrace();
    }

} catch (SQLException e) {
    e.printStackTrace();
}

}
```

Nota: Es conveniente planificar bien la aplicación para minimizar el tiempo en el que se tengan transacciones abiertas ejecutándose, ya que consumen recursos y suponen bloqueos en la base de datos que puede parar otras transacciones. En muchos casos, un diseño cuidadoso puede evitar usos innecesarios que se salgan fuera del modo estándar AutoCommit.

Hay una documentación muy extensa para programar con PL-SQL: procedimientos, funciones, triggers, etc., en el siguiente enlace:

Programación con [PL-SQL](#).

http://apuntesduoc.pbworks.com/w/file/fetch/54222708/040628_PLSQL_Basico.pdf

Interesante tutorial sobre transacciones y otras cuestiones con [MySQL](#).

https://www.uv.es/~jgutierrez/MySQL_Java/CursoMysql_ses2.pdf

Consultas al diccionario de datos

La Interfaz `DatabaseMetaData` (de la API `java.sql`) permite obtener Información sobre la base de datos en su conjunto.

Algunos métodos de la interfaz `DatabaseMetaData` devuelven listas de información en forma de objetos `ResultSet`, que se pueden recorrer en una repetitiva para obtener los metadatos. Para ello utilizaremos metodos `getString`, `getInt`, etc. Consulta la documentación de la [API JDBC](https://docs.oracle.com/javase/8/docs/api/java/sql/DatabaseMetaData.html) (<https://docs.oracle.com/javase/8/docs/api/java/sql/DatabaseMetaData.html>).

Algunos métodos `DatabaseMetaData` tienen argumentos que son patrones de cadena. Es posible usar patrones dentro de estas cadenas, "%" representa cualquier subcadena de 0 o más caracteres, y "_" representa un carácter. Algunos métodos útiles son:

- ✓ `getColumns(String catalog, String schemaPattern, String tableNamePattern, String columnNamePattern)`
Devuelve en un `ResultSet` la descripción de las columnas de la tabla disponibles en el catálogo especificado.
- ✓ `getDriverName()`
Devuelve en un `String`, el nombre del controlador JDBC.
- ✓ `getDriverVersion()`
Devuelve en un `String`, el número de versión de este controlador JDBC como a `String`.
- ✓ `getExportedKeys(String catalog, String schema, String table)`
Devuelve en un `ResultSet` la descripción de las columnas de clave Foranea que hacen referencia a las columnas de clave principal de la tabla dada.
- ✓ `getFunctions(String catalog, String schemaPattern, String functionNamePattern)`
Devuelve en un `ResultSet` la descripción del sistema y las funciones de usuario disponibles en el catálogo dado.
- ✓ `getMaxConnections()`
Recupera el número máximo de conexiones simultáneas a esta base de datos que son posibles.
- ✓ `getPrimaryKeys(String catalog, String schema, String table)`
Devuelve en un `ResultSet` una descripción de las columnas de clave primaria de la tabla dada.
- ✓ `getProcedures(String catalog, String schemaPattern, String procedureNamePattern)`
Recupera una descripción de los procedimientos almacenados disponibles en el catálogo dado.
- ✓ `getSchemas()`
Devuelve en un `ResultSet` los nombres de esquema disponibles en esta base de datos.

- ✓ `getSQLKeywords()`
Recupera una lista separada por comas de todas las palabras clave de SQL de esta base de datos que NO son también palabras clave de SQL: 2003.
- ✓ `getStringFunctions()`
Recupera una lista separada por comas de funciones de cadena disponibles con esta base de datos.
- ✓ `supportsAlterTableWithAddColumn()`
Recupera si esta base de datos admite ALTER TABLE con añadir columna. Devuelve un booleano
- ✓ `supportsAlterTableWithDropColumn()`
Recupera si esta base de datos es posible borrar una columna con ALTER TABLE. Devuelve un booleano.
- ✓ `supportsTransactions()`
Recupera si esta base de datos admite transacciones.

Ejemplo:

```
public class EjemploMetadata {
    public static void main(String[] args) {
        try {
            Connection conexion = DriverManager.getConnection("jdbc:mysql://localhost/ejemplo", "ejemplo", "ejemplo");
            java.sql.DatabaseMetaData dbmd = conexion.getMetaData(); // Creamos objeto DatabaseMetaD
            ata

            ResultSet resul = null;
            String nombre = dbmd.getDatabaseProductName();
            String driver = dbmd.getDriverName();
            String url = dbmd.getURL();
            String usuario = dbmd.getUserName();
            System.out.println("INFORMACIÓN SOBRE LA BASE DE DATOS:");
            System.out.printf("Nombre : %s %n", nombre);
            System.out.printf("Driver : %s %n", driver);
            System.out.printf("URL      : %s %n", url);
            System.out.printf("Usuario: %s %n", usuario);
            // Obtener información de las tablas y vistas que hay
            resul = dbmd.getTables(null, "ejemplo", null, null);
            while (resul.next()) {
                String catalogo = resul.getString(1); // columna 1
```

```
        String esquema = resul.getString(2); // columna 2
        String tabla = resul.getString(3); // columna 3
        String tipo = resul.getString(4); // columna 4
        System.out.printf("%s - Catalogo: %s, Esquema: %s, Nombre: %s %n", tipo, cata
logo, esquema, tabla);
    }
    conexion.close(); // Cerrar conexion
} catch (SQLException e) {
    System.out.println(e.getMessage());
    System.out.println(e.getErrorCode());
    System.out.println(e.getSQLState());
}
}
}
```

Recuperación y modificación de valores de ResultSet

Un objeto [ResultSet](#) es una tabla de datos que representa un conjunto de resultados de base de datos. Puede ser creado a través de cualquier objeto que implementa el interfaz [Statement](#), incluyendo [PreparedStatement](#), [CallableStatement](#).

Es necesario un bucle para iterar a través de todos los datos en el [ResultSet](#), utilizando el método [ResultSet.next](#) se hace avanzar al cursor a la siguiente fila.

Los objeto [ResultSet](#) predeterminados no permiten la actualización , además, solo se puede mover el cursor hacia adelante. Sin embargo, puede crear objetos [ResultSet](#) que se pueden desplazar hacia adelante y hacia atrás o moverse a una posición absoluta y actualizarse.

Para modificar valores directamente desde el [ResultSet](#), este debe ser:

```
Statement stmt = conexion.createStatement(  
    ResultSet.TYPE_SCROLL_SENSITIVE, // permite moverse y ver cambios  
    ResultSet.CONCUR_UPDATABLE      // permite actualizar datos  
);
```

Además, es necesario actualizar el dato en la fila actual del objeto [ResultSet](#):

```
resultSet.updateFloat( "PRICE", f * percentage); //recupera la columna con nombre "PRICE"  
resultSet.updateRow();
```

Veamos el ejemplo siguiente, el método [modifyPrices \(float percentage\)](#) multiplica el precio del café por un porcentaje dentro de la repetitiva que recorre, utilizando el objeto [ResultSet](#), todas las filas de la tabla [coffees](#) utilizando el objeto [ResultSet](#).

Ejemplo:

```
public void modifyPrices(Connection con, float percentage) {  
    String sql = "SELECT * FROM COFFEES";  
  
    try {  
        Statement stmt = con.createStatement(  
            ResultSet.TYPE_SCROLL_SENSITIVE,  
            ResultSet.CONCUR_UPDATABLE  
        );  
        ResultSet uprs = stmt.executeQuery(sql)  
    } {
```

```
while (uprs.next()) {
    float precio = uprs.getFloat("PRICE");
    float nuevoPrecio = precio * percentage;

    uprs.updateFloat("PRICE", nuevoPrecio);
    uprs.updateRow();

    System.out.printf("Precio actualizado: %.2f → %.2f%n", precio, nuevoPrecio);
}

} catch (SQLException e) {
    e.printStackTrace(); // o uso de una utilidad para imprimir detalles
}
}
```

Para saber más de los tipos de [ResultSet](#) que se pueden utilizar en [Oracle](#), visita:

https://docs.oracle.com/javase/tutorial/jdbc/basics/retrieving.html#rs_update

Ejemplo completo de uso ResultSet para actualización.

¿Qué hace este programa?

- Conecta a una base de datos Oracle (usando JDBC).
- Lee la tabla Employees, mostrando su contenido.
- Incrementa el salario de todos los empleados en +5.
- Inserta un nuevo empleado con datos fijos.
- Elimina la segunda fila del ResultSet.
- Muestra los resultados tras cada operación.

```
import java.sql.*;
import java.time.LocalDateTime;

public class JDBCExample {

    // URL de conexión y credenciales
    static final String DB_URL = "jdbc:oracle:thin:@localhost:1521:XE";
    static final String USER = "scott";
```

```
static final String PASS = "tiger";

public static void main(String[] args) {

    try (
        Connection conn = DriverManager.getConnection(DB_URL, USER, PASS);
        Statement stmt = conn.createStatement(ResultSet.TYPE_SCROLL_INSENSITIVE, ResultSet.CONCUR_UPDA
TABLE);
        ResultSet rs = stmt.executeQuery("SELECT EMPLOYEE_ID, first_name, last_name, salary, email, hi
re_date FROM Employees")
    ) {
        System.out.println("Conectado a la base de datos.\n");

        // Mostrar datos actuales
        System.out.println("Antes de la actualización:");
        printRs(rs);

        // Reposicionar antes del primer registro
        rs.beforeFirst();

        // Aumentar salario de todos
        while (rs.next()) {
            double salary = rs.getDouble("salary");
            rs.updateDouble("salary", salary + 5);
            rs.updateRow();
        }

        System.out.println("Después del incremento de salario:");
        rs.beforeFirst();
        printRs(rs);

        // Insertar nuevo empleado
        System.out.println("Insertando nuevo registro...");
        rs.moveToInsertRow();
        rs.updateInt("EMPLOYEE_ID", 300);
        rs.updateString("first_name", "John");
        rs.updateString("last_name", "Paul");
    }
}
```

```
rs.updateInt("salary", 40);
rs.updateString("email", "ana@ana");
rs.updateTimestamp("hire_date", Timestamp.valueOf(LocalDateTime.now()));
rs.insertRow();

System.out.println("Después de la inserción:");
rs.beforeFirst();
printRs(rs);

// Eliminar el segundo registro
if (rs.absolute(2)) {
    System.out.println("Eliminando segundo registro:");
    printCurrentRow(rs);
    rs.deleteRow();
}

System.out.println("Después del borrado:");
rs.beforeFirst();
printRs(rs);

} catch (SQLException e) {
    e.printStackTrace();
}

System.out.println("Proceso finalizado.");
}

public static void printRs(ResultSet rs) throws SQLException {
    while (rs.next()) {
        printCurrentRow(rs);
    }
    System.out.println();
}

private static void printCurrentRow(ResultSet rs) throws SQLException {
    int id = rs.getInt("EMPLOYEE_ID");
    String fname = rs.getString("first_name");
```



```
String lname = rs.getString("last_name");  
double salary = rs.getDouble("salary");  
System.out.printf("EMPLOYEE_ID: %d, Nombre: %s %s, Salario: %.2f%n", id, fname, lname, salary);  
}  
}
```

Pool de conexiones

Acabamos de ver cómo se realiza una conexión a una base de datos. En ocasiones, sobre todo cuando se trabaja en el ámbito de las aplicaciones distribuidas, en entornos web, es recomendable gestionar las conexiones de otro modo.

Antes de ver en qué consiste, veamos cómo funciona la creación de conexiones en una aplicación clásica de escritorio **cliente-servidor** y luego veamos los problemas que se seguirán con dicha estructura en una **aplicación web**.

Problemas en la creación de conexiones

En una aplicación de escritorio se crea una conexión de base de datos al iniciar la aplicación y se cierra al finalizar la aplicación. Es decir que cada usuario que inicia la aplicación tiene una conexión en exclusiva para él. Y obviamente sería imposible compartirlas ya que cada aplicación estará en un ordenador independiente.

En una aplicación Web podríamos seguir un esquema similar, en el que cada usuario nuevo que se conecta a nuestra aplicación se le crea una conexión y al salir de la aplicación que se cierre su conexión. Esto que aparentemente es sencillo tiene unos problemas debido a la diferente naturaleza de las aplicaciones de escritorio y las web.

Si siguiéramos el mismo patrón de creación de conexiones de aplicaciones de escritorio en aplicaciones web acabaríamos con una cantidad enorme de conexiones activas (debido al gran número de usuarios) y con gran cantidad de conexiones abiertas sin usar (debido a usuarios que abandonan el portal y no lo hemos detectado).

Consecuencia de lo anterior al tener tantas conexiones a la base de datos se acabaría cayendo el servidor de base de datos debido a los recursos consumidos por todas las conexiones.

Podemos pensar que nuestro servidor puede aguantar todas esas conexiones ya que podemos tener pocos usuarios pero no suele ser así debido a:

- ✓ Si no se cierran las conexiones aun teniendo pocos usuarios es probable que acabemos saturando al servidor de conexiones sin usar.
- ✓ Las aplicaciones Web pueden tener picos de mucho tráfico, donde sería normal que se excediera la capacidad de nuestro servidor.
- ✓ Sería muy sencillo hacernos un ataque de denegación de servicio y saturando el servidor haciendo que se crearan gran cantidad de conexiones que no se usen.

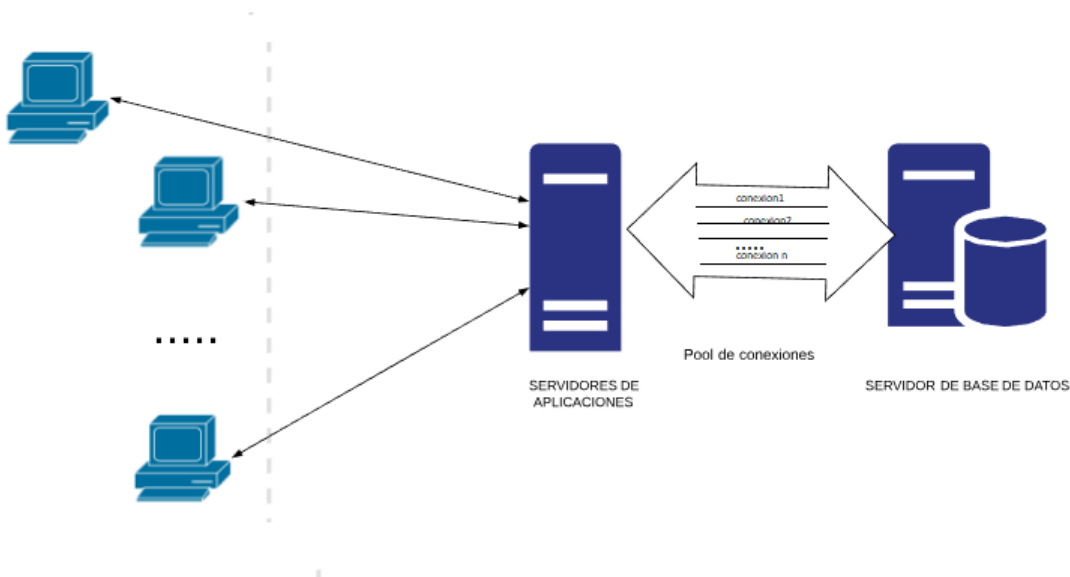
Una solución que nos evitaría las conexiones sin usar sería que se creara la conexión al iniciar cada petición web y se cerrara al finalizar dicha petición web. ¿El problema de eso? Crear y cerrar una conexión es muy costoso. Lo que tendríamos es una aplicación lentísima.

Pool de conexiones

El concepto de Pool de conexiones se ha estandarizado desde la versión 3.0 de [JDBC](#).

La solución del pool de conexiones tiene que solucionar los siguientes problemas:

- ✓ No tener tantas conexiones como usuarios ya que el número de usuarios es demasiado elevado.
- ✓ Cerrar la conexión.



Funcionamiento:

El servidor web tiene “n” conexiones ya creadas y conectadas a la base de datos (Se llaman conexiones esperando).

- ✓ Cuando llegan “m” peticiones web, la aplicación pide “m” conexiones al pool de conexiones, quedando esperando en el pool “n-m” conexiones. Esta operación es muy rápida ya que la conexión ya está creada y solo hay que marcarla como que alguien la está usando. Ahora hay “m” conexiones activas que está usando la aplicación.
- ✓ Cuando las peticiones web finalizan, las conexiones no se cierran sino que se devuelven al pool indicándole que ya se han acabado de usar las conexiones. Ahora vuelve a quedar “n” conexiones esperando en el pool. Esta operación también es muy rápida ya que realmente no se cierra ninguna conexión sino que simplemente se marcan como que ya no las están usando nadie.

Si se piden más conexiones de las que hay esperando en el pool se crearán en ese instante nuevas conexiones hasta el máximo de conexiones que permita el pool.

Al devolver una conexión al pool, ésta se queda esperando para que otra petición la pueda usar. Si hay ya demasiadas conexiones esperando a ser usadas se cerrarán para ahorrar recursos en el servidor de base de datos.

¿Qué hemos conseguido con el pool?

- ✓ Ahora las conexiones ya no se quedarán abiertas cuando el usuario se marcha del portal ya que cada conexión se pseudo-abre y pseudo-cierra con cada petición.
- ✓ No tenemos tantas conexiones como usuarios usan la aplicación ya que solo se necesitan tantas como usuarios hay haciendo una petición en ese instante. Pensemos por un momento en facebook. ¿cuandos usuarios están conectados a facebook? Supongamos “x”. Pero ¿cuantos están realmente haciendo una petición y no viendo los datos que se han servido? Supongamos “y”. Obviamente “y” es mucho menor que “x”. Con lo que nos hemos ahorrado “x-y” conexiones.
- ✓ Al iniciar un servidor Java EE, automáticamente el pool de conexiones crea un número de conexiones físicas iniciales.

El uso de bibliotecas como:

- ✓ Apache Commons DBCP: Es una de las bibliotecas más utilizadas para la creación de pools de conexiones en aplicaciones Java.
- ✓ HikariCP : Es un pool de conexiones muy eficiente, conocido por su bajo consumo de recursos y alta velocidad.
- ✓ Tomcat JDBC Pool: El pool de conexiones usado en los servidores Apache Tomcat.

Nos permiten gestionar de manera eficiente las conexiones, reduciendo la sobrecarga y garantizando que las aplicaciones puedan escalar adecuadamente sin saturar los recursos de la base de datos.

Cuando un objeto Java del servidor J2EE necesita una conexión, la solicita a través del método `dataSource.getConnection()`, la fuente de datos `javax.sql.DataSource` habla con el pool de conexiones y éste le entrega una conexión lógica `java.sql.Connection`. Esta conexión lógica la recibe por último, el objeto Java.

Cuando un objeto Java del servidor Java EE desea cerrar una conexión a través del método `connection.close()`, la fuente de datos `javax.sql.DataSource` habla con el pool de conexiones y le devuelve la conexión lógica en cuestión.

Si hay un pico en la demanda de conexiones a la base de datos, el pool de conexiones de forma transparente crea más conexiones físicas de objetos tipo `Connection`. Si por el contrario las conexiones a la base de datos disminuyen, el pool de conexiones, también de forma transparente elimina conexiones físicas de objetos de tipo `Connection`.

En el siguiente enlace puedes ver un ejemplo paso a paso con `NetBeans` y `MySQL`, sobre la creación de un pool de conexiones.

<https://old.chuidiang.org/java/mysql/BasicDataSource-Pool-Conexiones.php>

Ejemplo de pool de conexiones Mysql

Para aprender a realizar un pool de conexiones con MySQL, se aporta el siguiente vídeo explicativo:

<https://www.youtube.com/watch?v=pBXeGbsxk5Q>

Ejemplo uso de un pool de conexiones en Java con Apache Commons DBCP

Apache Commons DBCP es una de las bibliotecas más utilizadas para la creación de pools de conexiones en aplicaciones Java.

1. Dependencia de Maven (para proyectos Maven):

Si estás usando Maven, añade la siguiente dependencia para Apache Commons DBCP en tu pom.xml:

```
<dependency>
  <groupId>org.apache.commons</groupId>
  <artifactId>commons-dbcp2</artifactId>
  <version>2.9.0</version> <!-- Usa la versión más reciente disponible -->
</dependency>
```

2. Código de ejemplo usando BasicDataSource:

En este ejemplo, crearemos un pool de conexiones utilizando BasicDataSource, que es una implementación de un pool de conexiones en Apache Commons DBCP.

```
import org.apache.commons.dbcp2.BasicDataSource;
import java.sql.Connection;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;

public class ConnectionPoolExample {

    private static BasicDataSource dataSource;
    // Inicialización del pool de conexiones
    static {
        dataSource = new BasicDataSource();
        dataSource.setUrl("jdbc:mysql://localhost:3306/tu_base_de_datos");
        dataSource.setUsername("tu_usuario");
        dataSource.setPassword("tu_contraseña");

        // Configuraciones del pool
```

```

dataSource.setMinIdle(5); // Número mínimo de conexiones ociosas en el pool
dataSource.setMaxIdle(10); // Número máximo de conexiones ociosas en el pool
dataSource.setMaxTotal(25); // Número máximo total de conexiones
dataSource.setMaxOpenPreparedStatements(100); // Número máximo de sentencias preparadas abiertas
}

public static Connection getConnection() throws SQLException {
    return dataSource.getConnection(); // Obtener una conexión del pool
}

public static void main(String[] args) {
    try (Connection connection = getConnection();
        Statement statement = connection.createStatement();
        ResultSet resultSet = statement.executeQuery("SELECT * FROM empleados")) {

        while (resultSet.next()) {
            System.out.println("Empleado ID: " + resultSet.getInt("id"));
            System.out.println("Nombre: " + resultSet.getString("nombre"));
        }
    } catch (SQLException e) {
        e.printStackTrace();
    }
}

```

Explicación del código anterior:

- Inicialización del pool de conexiones:

En el bloque estático (static {}), se configura el pool de conexiones usando BasicDataSource.

Se definen las propiedades del pool, como la URL de la base de datos, el usuario y la contraseña.

También se configuran parámetros como el número mínimo y máximo de conexiones ociosas y el número máximo total de conexiones.

- Método getConnection():

Este método proporciona una conexión desde el pool cada vez que es llamado. Si no hay conexiones disponibles, el pool creará una nueva (hasta llegar al número máximo definido).

- Uso de la conexión:

En el main(), se obtiene una conexión del pool usando el método getConnection().

Se realiza una consulta a la base de datos (SELECT * FROM empleados) y se procesan los resultados.

Después de usar la conexión, esta se devuelve automáticamente al pool cuando el bloque try-with-resources se cierra.

- Configuración del pool:
 - ✓ `setMinIdle(5)`: Define que el pool debe mantener al menos 5 conexiones ociosas en todo momento. Esto significa que incluso si no se están usando, el pool mantendrá 5 conexiones abiertas y listas para su uso.
 - ✓ `setMaxIdle(10)`: Define que el pool puede tener hasta 10 conexiones ociosas. Si hay más de 10 conexiones ociosas, el pool comenzará a cerrarlas para liberar recursos.
 - ✓ `setMaxTotal(25)`: Define el número máximo total de conexiones abiertas. Si 25 conexiones están en uso, las peticiones adicionales esperarán hasta que una conexión esté disponible o falle con una excepción de límite alcanzado.
 - ✓ `setMaxOpenPreparedStatements(100)`: Define el número máximo de sentencias preparadas que pueden estar abiertas al mismo tiempo.

Ejemplo de pool de conexiones Oracle

```
package PoolConexiones;

/*
DESCRIPTION

The code sample shows how to use the DataSource API to establish a connection
to the Database. You can specify properties with "setConnectionProperties".
This is the recommended way to create connections to the Database.
Note that an instance of oracle.jdbc.pool.OracleDataSource doesn't provide
any connection pooling. It's just a connection factory. A connection pool,
such as Universal Connection Pool (UCP), can be configured to use an
instance of oracle.jdbc.pool.OracleDataSource to create connections and
then cache them.

Step 1: Enter the Database details in this file.
        DB_USER, DB_PASSWORD and DB_URL are required

Step 2: Run the sample with "ant DataSourceSample"
NOTES
    Use JDK 1.7 and above
MODIFIED    (MM/DD/YY)
nbsundar    02/17/15 - Creation
*/

import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
```

```
import java.util.Properties;
import oracle.jdbc.pool.OracleDataSource;
import oracle.jdbc.OracleConnection;
import java.sql.DatabaseMetaData;

public class DataSourceSample {
    // The recommended format of a connection URL is the long format with the
    // connection descriptor.
    final static String DB_URL= "jdbc:oracle:thin:@localhost:1521:XE";
    final static String DB_USER = "scott";
    final static String DB_PASSWORD = "tiger";
    /*
     * The method gets a database connection using
     * oracle.jdbc.pool.OracleDataSource. It also sets some connection
     * level properties, such as,
     * OracleConnection.CONNECTION_PROPERTY_DEFAULT_ROW_PREFETCH,
     * OracleConnection.CONNECTION_PROPERTY_THIN_NET_CHECKSUM_TYPES, etc.,
     * There are many other connection related properties. Refer to
     * the OracleConnection interface to find more.
     */
    public static void main(String args[]) throws SQLException {
        Properties info = new Properties();
        info.put(OracleConnection.CONNECTION_PROPERTY_USER_NAME, DB_USER);
        info.put(OracleConnection.CONNECTION_PROPERTY_PASSWORD, DB_PASSWORD);
        info.put(OracleConnection.CONNECTION_PROPERTY_DEFAULT_ROW_PREFETCH, "20");
        info.put(OracleConnection.CONNECTION_PROPERTY_THIN_NET_CHECKSUM_LEVEL, "REQUIRED");
        OracleDataSource ods = new OracleDataSource();
        ods.setURL(DB_URL);
        ods.setConnectionProperties(info);
        // With AutoCloseable, the connection is closed automatically.
        try (OracleConnection connection = (OracleConnection) ods.getConnection()) {
            // Get the JDBC driver name and version
            DatabaseMetaData dbmd = connection.getMetaData();
            System.out.println("Driver Name: " + dbmd.getDriverName());
            System.out.println("Driver Version: " + dbmd.getDriverVersion());
            // Print some connection properties
            System.out.println("Default Row Prefetch Value is: " + connection.getDefaultRowPrefetch());
        }
    }
}
```



```
        System.out.println("Database Username is: " + connection.getUserName());
        System.out.println();
        // Perform a database operation
        printEmployees(connection);
    }
}
/*
 * Displays first_name and last_name from the employees table.
 */
public static void printEmployees(OracleConnection connection) throws SQLException {
    // Statement and ResultSet are AutoCloseable and closed automatically.
    try (Statement statement = connection.createStatement()) {
        try (ResultSet resultSet = statement
            .executeQuery("select first_name, last_name from employees")) {
            System.out.println("FIRST_NAME" + " " + "LAST_NAME");
            System.out.println("-----");
            while (resultSet.next())
                System.out.println(resultSet.getString(1) + " " + resultSet.getString(2) + " ");
        }
    }
}
```

Operaciones: ejecución de consultas

Para operar con una base de datos, ejecutando las consultas necesarias, nuestra aplicación deberá hacer:

- **Cargar** el driver necesario para comprender el protocolo que usa la base de datos en cuestión.
- **Establecer** una conexión con la base de datos.
- **Enviar** consultas SQL y procesar el resultado.
- **Liberar** los recursos al terminar.
- **Gestionar** los errores que se puedan producir.

Podemos utilizar los siguientes tipos de sentencias:

- **Statement**: para sentencias sencillas en SQL.
- **PreparedStatement**: para consultas preparadas, como por ejemplo las que tienen parámetros.
- **CallableStatement**: para ejecutar procedimientos almacenados en la base de datos.

El API JDBC distingue dos tipos de consultas:

- **Consultas**: SELECT
- **Actualizaciones**: INSERT, UPDATE, DELETE, sentencias DDL.

Excepciones y cierre de conexiones

Debemos tener en cuenta siempre que las conexiones con una base de datos consumen muchos recursos en el sistema gestor, y por lo tanto en el sistema informático en general. Por ello, conviene cerrarlas con el método `close` siempre que vayan a dejar de ser utilizadas, en lugar de esperar a que el `garbage collector` de Java las elimine.

También conviene cerrar las consultas (`Statement` y `PreparedStatement`) y los resultados (`ResultSet`) para liberar los recursos.

Una excepción es una situación que no se puede resolver y que provoca la detención del programa de manera abrupta. Se produce por una condición de error en tiempo de ejecución.

En Java hay muchos tipos de excepciones, el paquete `java.lang.Exception` es el que contiene los tipos de excepciones.

Aquí puedes ver un resumen esquematizado de conceptos vistos en este tema y más:

<https://adictosaltrabajo.com/2011/02/25/tutorial-basico-jdbc/>

Excepciones

Cuando se produce un error durante la ejecución de un programa, se genera un objeto asociado a esa excepción. Ese objeto es de la clase `Exception` o de alguna de sus subclases. Este objeto se pasa entonces al código que se ha definido para gestionar la excepción.

En una porción de programa donde se trabajara con ficheros, y con bases de datos podríamos tener esta estructura para capturar las posibles excepciones.

```
try {  
    // Bloque de instrucciones del try  
  
} catch (FileNotFoundException fnfe) {  
    // Bloque para excepción por fichero no encontrado  
  
} catch (IOException ioe) {  
    // Bloque para excepción por error de entrada salida  
  
} catch (SQLException sqle) {  
    // Bloque para excepción por error con SQL  
} catch (Exception e) {  
  
} finally {  
    // Instrucciones finales para, por ejemplo, limpieza  
}
```

El bloque de instrucciones del `try` es el que se ejecuta, y si en él ocurre un error que dispara una excepción, entonces se mira si es de tipo fichero no encontrado; si es así, se ejecutarían las instrucciones del bloque del fichero no encontrado. Si no era de ese tipo la excepción, se mira si es del siguiente tipo, o sea, de entrada salida, y así sucesivamente.

Las instrucciones que hay en el bloque del `finally`, se ejecutarán siempre, se haya producido una excepción o no, ahí suelen ponerse instrucciones de limpieza, de cierre de conexiones, etc.

Las acciones que se realizan sobre una base de datos pueden lanzar la excepción `SQLException`. Este tipo de excepción proporciona entre otra información:

- Una cadena de caracteres describiendo el error. Se obtiene con el método `getMessage`.
- Un código entero de error que especifica al fabricante de la base de datos.

Para saber más sobre excepciones, puedes consultar este enlace

<https://elvex.ugr.es/decsai/java/pdf/B2-excepciones.pdf>

Cierre de conexiones

Como ya hemos dicho, al trabajar con bases de datos, se consumen muchos recursos por parte del sistema gestor, así como del resto de la aplicación.

Por esta razón, resulta totalmente conveniente cerrarlas con el método `close` cuando ya no se utilizan.

Podríamos por tanto tener un ejemplo de cómo hacer esto:

```
Connection con = null ;
try {

    con = DriverManager.getConnection("jdbc:odbc:admdb");
    System.out.println("Conexión realizada con éxito.");
    // Aquí hacemos lo que necesitemos
}
catch( SQLException e ) {

    // Aquí haríamos lo necesario para gestionar la excepción SQL
}

finally {
    // Si hay conexión la cerramos
    if( con != null ) {
        try { con.close(); }
        catch( SQLException e ) {
            System.out.println(e.getMessage());
        }
    }
}
```

Mejor utilizar Try-with-resources, ver apartado donde se habla de cómo se cierran automáticamente las conexiones.

EJERCICIOS BLOQUE 1 (SQL – DDL)

A partir del siguiente ejemplo <https://dev.mysql.com/doc/employee/en/sakila-structure.html>

1. Crea las tablas:
 - a. Empleado
(id, nombre varchar(20), fecha_nacimiento DATE, genero tinyint(1), departamento_id)
 - b. Departamento
(id, nombre varchar(20), ubicación varchar(100))
2. Actualiza la tabla empleado
 - a. Añade la columna departamento_id y crea la Foreign Key con departamento
 - 1 departamento muchos empleados
 - 1 empleado 1 departamento

EJERCICIOS BLOQUE 2 (SQL – DML)

Siguiendo el ejemplo anterior <https://dev.mysql.com/doc/employee/en/sakila-structure.html>

1. Inserta datos en las diferentes tablas, todos los datos deben ir por parámetros y debe utilizarse el PreparedStatement para evitar SQLInjection
 - Tabla departamento (el id se debe de autogenerar)
 - 1, contabilidad, PBO
 - 2, marketing, PBO,
 - 3, RRHH, PB1
 - Tabla Empleado (el id se debe de autogenerar)
 - 1, Alvaro, 25/11/1979, 1, 1
 - 2, Juan, 07/07/2001, 1, 2
 - 3, Marta, 25/12/1997, 0, 3
 - 4, Silvia, 01/04/1992, 0, 2
2. Recupera los datos de los empleados junto con el nombre del departamento al cual pertenecen
3. Recupera todos los usuarios ordenados por fecha de nacimiento, de mayor a menor
4. Edita la fecha de nacimiento del empleado con nombre Marta y establece "04/03/2002"
5. Elimina el usuario con el id 1

PRACTICA FINAL 1

1. Se trata de crear una base de datos de una aerolínea utilizando la herramienta MySQLWorkBench. La base de datos se llamará **vuelospasajeros**.
2. Consiste en crear dos de sus tablas VUELOS y PASAJEROS .

A continuación la definición de cada tabla:

- Tabla VUELOS
 - Campos:
 - COD_VUELO VARCHAR(10)
 - HORA_SALIDA VARCHAR(15)
 - DESTINO VARCHAR(15)
 - PROCEDENCIA VARCHAR(15)
 - PLAZAS_FUMADOR int(3)
 - PLAZAS_NO_FUMADOR int(3)
 - PLAZAS_TURISTA int(3)
 - PLAZAS_PRIMERA int(3)
 - La clave primaria de esta tabla estará formada por el campo COD_VUELO.
- Tabla PASAJEROS
 - Campos:
 - NUM int(7),
 - COD_VUELO VARCHAR(10) – Este campo será foreign key de la tabla VUELOS campo COD_VUELO
 - TIPO_PLAZA VARCHAR(2) -- 'TU': TURISTA, 'PR': PRIMERA
 - FUMADOR VARCHAR(2) -- 'SI' O 'NO'
 - La clave primaria de esta tabla se llamará PK_PASAJEROS y estará formada por los campos NUM y COD_VUELO.

3. Consiste en añadir registros a ambas tablas.

A continuación los registro a añadir a cada tabla:

- Tabla VUELOS

COD_VUELO : IB-SP-4567
HORA_SALIDA : 27/03/99-10:30
DESTINO : PARIS
PROCEDENCIA : MADRID
PLAZAS_FUMADOR : 25
PLAZAS_NO_FUMADOR : 100
PLAZAS_TURISTA: 160
PLAZAS_PRIMERA: 40

COD_VUELO : IB-BA-46DC
HORA_SALIDA : 28/03/99-12:30
DESTINO : ROMA
PROCEDENCIA : MADRID
PLAZAS_FUMADOR : 90
PLAZAS_NO_FUMADOR : 100
PLAZAS_TURISTA: 160
PLAZAS_PRIMERA: 30

- Tabla PASAJEROS

NUM: 123
COD_VUELO: IB-SP-4567
TIPO_PLAZA: TURISTA
FUMADOR : Si

NUM: 126
COD_VUELO: IB-BA-46DC
TIPO_PLAZA: TURISTA
FUMADOR : Si

NUM: 124
COD_VUELO: IB-SP-4567
TIPO_PLAZA: PRIMERA
FUMADOR : Si

NUM: 127
COD_VUELO: IB-BA-46DC
TIPO_PLAZA: PRIMERA
FUMADOR : Si

NUM: 125
COD_VUELO: IB-SP-4567
TIPO_PLAZA: PRIMERA
FUMADOR : NO

NUM: 128
COD_VUELO: IB-BA-46DC
TIPO_PLAZA: PRIMERA
FUMADOR : Si

PRACTICA FINAL 2

Se trata de hacer una aplicación en Java que acceda a la base de datos MySQL de una aerolínea, creada anteriormente. Consiste en una conexión a las tablas VUELOS y PASAJEROS de una BD MySQL que se implementara con el fichero que se adjunta en caso de no haberla podido crear en la práctica anterior “[createBBDD_vuelospasajeros.sql](#)”.

- Mostrar y pedir información de la base de datos en general. Muestra las 3 primeras columnas de 2 filas de la tabla vuelos .
- Mostrar la información de la tabla pasajeros.
- Ver la información de los pasajeros de un vuelo, pasando el código de vuelo como parámetro.
- Insertar un vuelo cuyos valores se pasan como parámetros:
COD_VUELO : FR-PR-5000
HORA: 21/05/99 15:30
DESTINO: Madrid
PROCEDENCIA: Paris
PLAZAS FUMADOR: 30
PLAZAS NO FUMADOR: 120
PLAZAS TURISTA: 90
PLAZAS PRIMERA: 60
- Borrar el vuelo que se metió anteriormente en el que se pasa por parámetro su número de vuelo.
- Modificar los vuelos de fumadores a no fumadores.

PRACTICA EXTRA

Realiza la siguiente practica.

1. Crea un programa que genere la siguiente estructura de packages y clases:

- Tema2
 - Common
 - Column
 - DDL
 - DML
 - Entities
 - Persona
 - Direccion
 - H2
 - Main
 - MySQL
 - Main

2. Define las clases Persona y Direccion:

Persona (id, nombre, teléfono,dirección,ArrayList<Direccion>)

Direccion(id, idPersona,dirección)

3. Define la clase Column, esta representa una columna de BD. Decide que variables debe contener para generar una nueva columna en la creación de una nueva tabla

Ejemplo: nombre VARCHAR(30) NOT NULL

4. Define la clase DDL, debe contener las funciones:

1. Crear tabla, debe recibir por parámetros los datos de la tabla (nombre y columna)

1. Crear ambas tablas

2. Actualizar tabla

1. Añade la FK entre Dirección y Persona

3. Eliminar tabla

1. Eliminar ambas tablas

5. Define la clase DML, debe contener las funciones

1. Consultar datos

1. Consultar 1 persona por id

2. Consultar todas las personas

3. Consultas 1 persona por id junto a sus direcciones

2. Insertar datos

1. Insertar persona

2. Insertar dirección

3. Actualizar datos

1. Modificar una persona

2. Modificar una dirección

4. Eliminar datos

1. Eliminar una persona y su dirección

6. Debe poder ejecutarse en MySQL y H2, por ello tenemos 2 Main diferentes